

A Mathematical Inspection Architecture for Full-ZB 3x3 Speedcubing

Color-Neutral Cross, Three F2L Pairs, ZBLS, and ZBLL
via Speffz State Transitions and Adjoint Recognition Probes

Central objective. During inspection, compile a deterministic six-macro solve

$$[C] [A_1] [A_2] [A_3] [Z] [L],$$

where C is a color-neutral cross, A_1, A_2, A_3 solve the first three F2L pairs, Z is the exact ZBLS algorithm for the final pair and edge orientation, and L is the exact ZBLL algorithm including required AUFs.

Concept initiated by: Jerich Lee
Document type: Technical framework and research specification
Version: 1.0
Date: September 1, 2026

This document distinguishes exact cube mathematics from proposed cognitive machinery. The universal ability to plan an entire ZB solve in less than 15 seconds has not been empirically demonstrated; it is the terminal research objective of the architecture developed here.

Abstract

This monograph develops a systematic inspection method for planning a complete ZB solve on a 3×3 Rubik’s Cube. The method begins with an exact Speffz-coordinate compiler for a color-neutral cross and the first three F2L pairs. Each chosen algorithm is treated not only as a motor sequence, but as a permutation operator acting on every tracked corner and edge sticker. This turns inspection into repeated state compilation rather than informal visual prediction.

The full-ZB extension introduces a bidirectional architecture. F2L pair state is propagated *forward*; only the last-layer information that becomes necessary at each gate is propagated *backward* as a recognition query. Three binary edge-orientation probes determine the ZBLS edge-orientation context. After the exact ZBLS algorithm is selected, a five-probe predictor—three oriented corner frames and two oriented edge probes—determines the exact future edge-oriented last layer. The final relative edge permutation is compressed to an element of the alternating group A_4 , represented by a two-symbol code with 12 possible values.

The resulting live symbolic load follows

$$8 \longrightarrow 6 \longrightarrow 4 \longrightarrow 2 \longrightarrow 3 \longrightarrow 5,$$

rather than requiring a persistent 14–16-letter full-cube memo. The document supplies the mathematical derivations, state-transition rules, Speffz generator maps, algorithm-card specification, training curriculum, software-trainer requirements, inspection timing architecture, validation tests, and open human-performance questions required to turn the idea into a rigorous long-term program.

Contents

Abstract	i
1 Objective, Scope, and Status	1
1.1 The terminal inspection objective	1
1.2 Competition boundary	1
1.3 What is established and what is proposed	2
2 Frames, Slots, and Speffz Coordinates	2
2.1 Color-neutral frame	2
2.2 Standard Speffz lettering	2
2.3 Exact F2L pair code	3
2.4 Algorithms as sticker permutations	3
3 Why the Speffz Pair Code Is Nearly Minimal	4
3.1 Information carried by three known pairs	4
3.2 Exact state versus canonical case	4
4 The True F2L State Space	5
4.1 Why the familiar 41 cases are insufficient	5
4.2 Normalization and extraction layer	5
4.3 Persistent pseudo-D phase	6
4.4 AUF and slot variants as gauge variables	6
5 Transition Maps and the Maps to Learn	7
5.1 Generator hierarchy	7
5.2 Trigger-level example	7
5.3 Algorithm card	7

5.4	Inspection-aware cost function	8
6	The Cross-Plus-Three Forward Compiler	8
6.1	State machine	8
6.2	Destructive compilation	8
6.3	Forward compiler procedure	9
6.4	Color-neutral cross selection	9
7	Residual-State Mathematics for Full ZB	9
7.1	Paired corner and edge sites	9
7.2	Relative corner-edge permutation	10
7.3	Residual-state counts	10
7.4	Contraction under solved pairs	11
7.5	Minimal image code for A_n	11
7.6	Why full forward tracking is not the preferred human method	11
8	The Adjoint Full-ZB Architecture	12
8.1	State liveness	12
8.2	Probe pullback identity	12
8.3	Prefix notation	12
9	The Three-Probe EO Sensor for ZBLS	13
9.1	Five unsolved edges at the ZBLS boundary	13
9.2	One-sticker orientation test	13
9.3	Adjoint EO query	14
9.4	Why identities are not needed at this gate	14
10	The Five-Probe Predictor for ZBLL	14
10.1	The exact post-ZBLS state	14
10.2	Three oriented corners determine the full corner state	14
10.3	Two oriented edges determine the full edge permutation	14
10.4	One Speffz token per corner frame	15
10.5	ZBLS predictor quintet	15
10.6	Sensor-basis variants	15
11	The A_4 Tetrahedral Edge Chunk	16
11.1	Relative edge permutation after ZBLS	16
11.2	Why there are 12 holistic edge chunks	16
11.3	Two-symbol code	16
11.4	Color-neutral relational observables	17
12	Complete Online Inspection Algorithm	17
12.1	Exact procedure	17
12.2	Compiler pseudocode	18
12.3	Core invariants	19
13	Algorithm-Set Design for Predictive Inspection	19
13.1	ZBLS card	19
13.2	Probe-overlap optimization	19
13.3	Payload-transparent macros	20
13.4	Deterministic policy	20
13.5	Superchunks	20

14 Holistic Human Representations	20
14.1 The F2L constellation	20
14.2 The EO triad	20
14.3 The predictor quintet	21
14.4 The tetrahedral edge token	21
14.5 Self-indexing motor chain	21
15 Training Curriculum	21
15.1 Error taxonomy	22
15.2 Training order for ZBLS and ZBLL	23
16 Software Trainer and Verification System	23
16.1 Required macro record	23
16.2 Drill modes	24
16.3 Property-based correctness tests	24
16.4 Metrics	24
16.5 Algorithm search objective	25
17 Inspection Timing and Fallback Architecture	25
17.1 Aspirational mature schedule	25
17.2 Fallback ladder	25
18 Feasibility, Limits, and Research Questions	26
18.1 Information-theoretic possibility	26
18.2 Why information theory is not the bottleneck	26
18.3 Symbolic-operation budget	26
18.4 Central empirical hypotheses	26
18.5 Honest conclusion on attainability	27
19 Implementation Roadmap	27
19.1 Phase I: exact simulator and cards	27
19.2 Phase II: fixed-frame proof of concept	27
19.3 Phase III: complete fixed-frame system	27
19.4 Phase IV: cognitive optimization	27
19.5 Phase V: adaptive and color-neutral system	27
19.6 Phase VI: universal test	28
20 Condensed Reference Architecture	28
A Standard Speffz Generator Maps	28
A.1 Whole-cube frame maps	29
B Detailed Derivations	29
B.1 Full cube state count	29
B.2 Post-three-pair suffix count	29
B.3 Post-ZBLS state and the 12 edge relations	29
C Quick Reference Sheet	29
D What “full mastery” would mean	32
D.1 The solve would change from reactive to compiled	32
D.2 Lookahead would almost disappear from the timed solve	32

D.3	Solve-time variance could fall dramatically	33
D.4	F2L would become a global optimization problem	34
D.5	Algorithm quality would acquire a second dimension	34
D.6	ZBLL would stop being 493 unrelated visual cases	35
D.7	Full color neutrality would extend through the entire solve	36
D.8	The cube would become external working memory	36
D.9	It would demonstrate extreme expert chunking	37
D.10	The solver would develop bidirectional mental simulation	37
D.11	The solve could become a self-indexing motor chain	38
D.12	Errors could be localized with compiler checkpoints	38
D.13	Training would separate into distinct engineering problems	39
D.14	It could create an entirely new software ecosystem	39
D.15	It could establish a new speedcubing research program	40
D.16	The method could generalize beyond CFOP and ZB	40
D.17	It would redefine what “lookahead” means	41
D.18	It could shift the speed limit from cognition to biomechanics	41
D.19	It would enable deliberate last-layer steering	41
D.20	The greatest benefit would probably be averages, not lucky singles	42
D.21	Full mastery would not imply several things	42
	The deepest implication	43
E	Mathematical and Algorithmic Analysis of AZBC	44
E.1	What kind of algorithm is AZBC?	44
E.2	The ordinary asymptotic answer: $\Theta(1)$	45
E.3	A useful parameterized model	45
E.4	Exact forward-compilation complexity	46
E.5	Complexity of the adjoint probes	47
E.6	Work–span analysis: the key parallelism result	48
E.7	Total online time complexity	48
E.8	AZBC as a shortest-path or dynamic-programming problem	50
E.9	Correctness theorem	50
E.10	Online space complexity	52
E.11	Logical visual-query complexity	53
E.12	A new bound on EO–quintet probe overlap	53
E.13	Information-theoretic analysis	54
E.14	The A_4 token as an optimal edge code	56
E.15	Offline preprocessing complexity	57
E.16	Direct-table baseline versus AZBC	58
E.17	Time–space tradeoffs from superchunks	59
E.18	Best-, average-, and worst-case behavior	59
E.19	AZBC as a hard-real-time and anytime algorithm	60
E.20	Error-propagation complexity	60
E.21	Formal algorithm profile	61
E.22	The central algorithmic conclusion	62
F	A fully worked AZBC example solve	63
F.1	Conventions	63
F.2	Scramble	64
F.3	Initial cross and F2L acquisition	64
F.4	Transporting all four pairs through the cross	65
F.5	Forward compilation of the first three pairs	66

F.6	Decoding the remaining pair	68
F.7	The first adjoint query: determining EO for ZBLS	68
F.8	Selecting the ZBLS algorithm	71
F.9	The second adjoint query: determining the future ZBLL	71
F.10	Reading the five ZBLL probes	73
F.11	Reconstructing the complete corner state	75
F.12	Reconstructing the complete edge permutation	76
F.13	Computing the A_4 edge token	77
F.14	Selecting ZBLL	78
F.15	The complete compiled solve	78
F.16	Execution checkpoints	78
F.17	What was held in working memory	79
F.18	Probe overlap in this solve	80
F.19	The complete inspection computation in compact form	80
F.20	What full mastery would change	81

1 Objective, Scope, and Status

1.1 The terminal inspection objective

The terminal goal is universal compilation of a complete ZB solution during legal 3x3 inspection. For every scramble, the solver should determine, before starting the timer,

$$\mathcal{M} = [C][A_1][A_2][A_3][Z][L],$$

with the following meanings:

- C : an exact cross sequence in a color-neutral frame;
- A_1, A_2, A_3 : exact algorithms for the first three F2L pairs, including setups, extractions, AUFs, slot choices, and any temporary pseudoslotting;
- Z : the exact ZBLS sequence that solves the fourth pair and orients the last-layer edges;
- L : the exact ZBLL sequence, including pre-AUF and post-AUF conventions.

This is stronger than ordinary “cross plus one” or “cross plus two” inspection. It also differs from memorizing a raw face-turn solution. The intended terminal representation is a sequence of six *motor chunks*; each chunk may contain several turns but is recalled as one learned action.

1.2 Competition boundary

Under the WCA Regulations current on the date of this document, a competitor is allotted *strictly less than 15 seconds* to inspect the puzzle and start the solve. The judge calls “8 SECONDS” and “12 SECONDS.” An inspection time of exactly 15.00 seconds receives a +2 penalty, and 17.00 seconds is a DNF. The cube may be picked up and reoriented during inspection, but moves are prohibited [1].

Internal operating target

A mature version of this system should not aim at the legal boundary itself. It should target a start near 14.0–14.2 seconds, leaving margin for hand placement, timer readiness, and occasional cognitive hesitation.

1.3 What is established and what is proposed

Table 1: Epistemic status of the major components.

Component	Status	Interpretation
Cube permutations, orientations, parity constraints, state counts	Exact mathematics	Standard cubie-coordinate and group-theoretic facts.
Speffz sticker coordinates	Established convention	A standardized blindsolving lettering scheme [2].
ZBLS and ZBLL case families	Established speedcubing methods	ZBLS solves last slot while orienting LL edges; ZBLL solves an edge-oriented LL [5, 6].
Forward pair-state compiler	Exact proposed formalization	Correct whenever the stored algorithm maps and observed piece states are correct.
Adjoint EO and ZBLL probes	Exact proposed formalization	The pullback identity is exact; the probe sufficiency proofs are mathematical.
Holistic human chunking in < 15 s	Unvalidated performance hypothesis	Plausible as a research target, but not demonstrated universally.

2 Frames, Slots, and Speffz Coordinates

2.1 Color-neutral frame

A color-neutral solve begins by selecting an oriented cube frame

$$\Phi = (D, F),$$

where D is the chosen cross color and F is the chosen front center. Once D is fixed, there are four possible front choices, so there are $6 \cdot 4 = 24$ oriented frames.

Define the four side sectors by

$$0 = F, \quad 1 = R, \quad 2 = B, \quad 3 = L,$$

with arithmetic in $\mathbb{Z}_4$. Define the four F2L slots in the same cyclic order:

$$0 = FR, \quad 1 = RB, \quad 2 = BL, \quad 3 = LF.$$

Every symbol used later is relative to Φ . Consequently, the mathematical tables are color neutral: changing cross color changes the interpretation of the face names, not the transition rules.

2.2 Standard Speffz lettering

Speffz assigns letters A through X to oriented corner-sticker positions and, independently, the same letters A through X to oriented edge-sticker positions. Faces are traversed in the order U, L, F, R, B, D , and stickers of a given piece type are lettered clockwise when looking directly at a face [2].

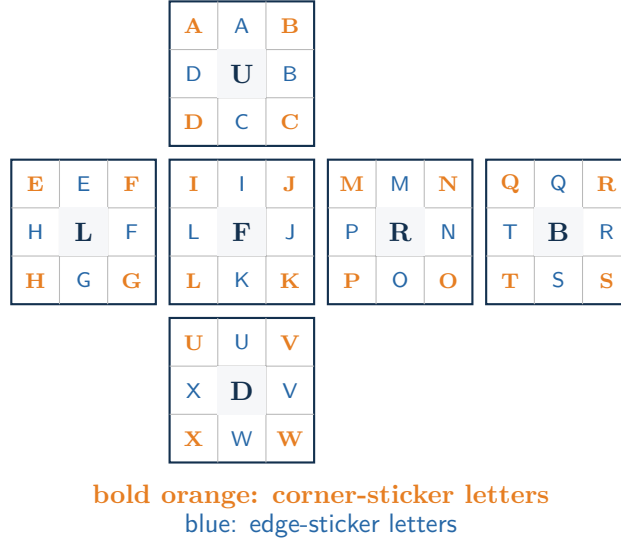


Figure 1: The standard Speffz net. The same letter alphabet is used separately for corner and edge sticker positions.

A letter denotes an *oriented sticker position*, not merely a cubie position. For example, the corner descriptions URF , RFU , and FUR refer to the same corner location with three distinct reference stickers. Likewise, UR and RU refer to the same edge location with opposite oriented stickers. This distinction is fundamental to blind-solving and is exactly what makes a Speffz letter carry orientation information [3, 4].

2.3 Exact F2L pair code

For each logical F2L pair $i \in \mathbb{Z}_4$, choose two distinguished stickers:

1. the D -colored sticker of its corner;
2. the leading side sticker of its edge: F for FR , R for RB , B for BL , and L for LF .

Define

$$q_i = (\gamma_i, \epsilon_i) \in \Sigma_C \times \Sigma_E,$$

where γ_i is the Speffz position occupied by the chosen corner sticker and ϵ_i is the Speffz position occupied by the chosen edge sticker.

Thus a code such as

$$q_i = SB$$

contains both cubie location and orientation for the known corner and edge identities.

For universal pair-order choice, store all four pairs in a fixed ring:

$$Q = [q_0, q_1, q_2, q_3].$$

A solved pair is replaced by $\perp$. A six-letter word such as SBTDKA is appropriate only after three pair identities and their order have already been fixed. The eight-letter ring is the preferred state before adaptive pair selection.

2.4 Algorithms as sticker permutations

Every exact algorithm A induces two permutations:

$$\tau_A^C : \Sigma_C \rightarrow \Sigma_C, \quad \tau_A^E : \Sigma_E \rightarrow \Sigma_E.$$

The pair update is

$$T_A(q_i) = \left(\tau_A^C(\gamma_i), \tau_A^E(\epsilon_i) \right).$$

For a word of three pair codes,

$$\begin{aligned} T_A(\gamma_1 \epsilon_1 \gamma_2 \epsilon_2 \gamma_3 \epsilon_3) \\ = \tau_A^C(\gamma_1) \tau_A^E(\epsilon_1) \tau_A^C(\gamma_2) \tau_A^E(\epsilon_2) \tau_A^C(\gamma_3) \tau_A^E(\epsilon_3). \end{aligned}$$

If A is executed and then B , the map of the combined sequence is

$$\tau_{AB} = \tau_B \circ \tau_A.$$

The map belongs to the *exact move sequence*. Two algorithms that solve the same visual F2L case may have completely different effects on every passive pair and on every future recognition probe.

3 Why the Speffz Pair Code Is Nearly Minimal

3.1 Information carried by three known pairs

Three known corner cubies can occupy ordered distinct positions in

$$(8)_3 = 8 \cdot 7 \cdot 6$$

ways and independently have 3^3 orientations. Three known edge cubies can occupy ordered distinct positions in

$$(12)_3 = 12 \cdot 11 \cdot 10$$

ways and have 2^3 orientations. Therefore the exact state count is

$$N_3 = (8)_3 3^3 (12)_3 2^3 = 95,800,320.$$

The corresponding information is

$$\log_2 N_3 \approx 26.51 \text{ bits.}$$

Six Speffz letters provide

$$6 \log_2 24 \approx 27.51 \text{ bits.}$$

Consequence

The six-letter code is within approximately one bit of an information-theoretically minimal exact code for three identified F2L pairs. A mixed-radix code could remove the redundancy, but would be far less suitable for visual acquisition and mental transformation.

3.2 Exact state versus canonical case

A complete inspection method must use two representations:

1. **Exact global state:** Speffz positions, sufficient to propagate pieces through arbitrary algorithms.
2. **Canonical local case:** a compressed recognition label used to choose an F2L or ZBLS algorithm.

The canonical case is a lossy quotient. Once a pair is reduced to “F2L case 17 in the FR slot,” absolute position, AUF, slot identity, and passive-piece information have been discarded. It is therefore unsuitable as the persistent transition state.

The inspection compiler repeatedly performs

$$\text{exact state} \longrightarrow \text{canonical case} \longrightarrow \text{exact chosen macro} \longrightarrow \text{new exact state.}$$

4 The True F2L State Space

4.1 Why the familiar 41 cases are insufficient

The familiar 41 F2L cases are complete only for the *last open slot*, when the other three F2L slots are occupied. They are not a complete classification of the first, second, or third pair because an active corner or edge may be buried in any still-open lower-layer position.

Let k be the number of already-solved F2L slots. The active corner can occupy

$$4 + (4 - k) = 8 - k$$

legal corner positions, with three orientations. The active edge can occupy $8 - k$ legal edge positions, with two orientations. The raw oriented pair-state count is therefore

$$R_k = 3(8 - k) \cdot 2(8 - k) = 6(8 - k)^2.$$

To count cases up to a U setup, apply Burnside’s lemma to the four-element group $\langle U \rangle$. A nonidentity U rotation fixes the pair only when both pieces are outside the U layer. The number fixed by each nonidentity element is

$$F_k = 3(4 - k) \cdot 2(4 - k) = 6(4 - k)^2.$$

Hence

$$n_k = \frac{R_k + 3F_k}{4}.$$

Table 2: Pair-state counts as F2L fills.

Solved slots k	Raw states R_k	U -normalized states n_k	Interpretation
0	384	168	First pair after cross
1	294	114	Second pair
2	216	72	Third pair
3	150	42	41 unsolved cases plus solved

Proposition 4.1. *The standard 41-case F2L set is the final member of the sequence*

$$168, \quad 114, \quad 72, \quad 42,$$

where the last count includes the already-solved case.

4.2 Normalization and extraction layer

A universal method should not learn 167, 113, and 71 unrelated extra algorithms. Instead, it should factor each nonstandard state into

$$\boxed{\text{safe extraction/setup} \circ \text{one of the 41 core cases}}.$$

For an exact pair state q , solved-slot mask m , target slot s , and phase variables, define a normalizer

$$\mathcal{N}(q; m, s, u, y, \delta) = (X, \kappa, A_\kappa),$$

where

- X is a safe setup or extraction sequence;
- κ is a canonical core-case label;
- A_κ is the selected exact core algorithm.

The executed macro is

$$A = X A_\kappa,$$

and the transition map attached to A is computed from the entire sequence, not from A_κ alone.

4.3 Persistent pseudo-D phase

The D layer has one physical phase

$$\delta \in \mathbb{Z}_4.$$

For a coherent persistent pseudo-F2L state, all corner-to-edge slot relations share this single cyclic shift:

$$\pi_\delta(i) = i + \delta \pmod{4}.$$

After a macro A ,

$$\delta_{k+1} = \delta_k + \Delta_D(A) \pmod{4}.$$

Thus three persistent pseudo-pairs do *not* create 4^3 independent physical phases. There is one global D -layer phase.

There are three distinct situations:

1. **Persistent coherent phase:** one δ is carried across macros.
2. **Temporary pseudoslotting:** the algorithm disturbs and restores D internally, so $\delta = 0$ at macro boundaries.
3. **Arbitrary mixed matching:** corner-edge pairings do not share a cyclic shift. The correct state is then a partial matching or permutation π , not three independent D offsets.

For a standard ZBLS interface, the cleanest contract is

$$\boxed{\delta = 0 \text{ before entering the ZBLS card.}}$$

4.4 AUF and slot variants as gauge variables

A U move between pairs is part of the exact macro. If absolute Speffz coordinates are maintained, no independent AUF variable is required: the letters already record the new positions.

A more compressed canonical implementation may store

$$u \in \mathbb{Z}_4, \quad y \in \mathbb{Z}_4$$

as explicit top-layer and side-frame gauges while keeping the local constellation normalized. These are two alternative coordinate styles:

- **Absolute mode:** update every Speffz letter; set $u = y = 0$.
- **Gauge-normalized mode:** keep a canonical local pattern and update u, y .

They must not be double-counted.

If A is a canonical FR-slot algorithm, its slot- s version is generated by conjugation:

$$A_s = y^s A y^{-s}.$$

The corresponding sticker map is generated by the same conjugation. Ergonomic back-slot algorithms that are not literal conjugates require their own exact cards.

5 Transition Maps and the Maps to Learn

5.1 Generator hierarchy

The complete sticker action of any algorithm can be generated from the six face-turn maps. The learning hierarchy should be:

1. six clockwise face-turn maps U, R, F, D, L, B ;
2. inverse and double turns derived by reversing or squaring cycles;
3. whole-cube frame maps x and y ;
4. common trigger maps, learned as single transformations;
5. direct macro maps for every exact F2L and extraction algorithm in the personal algset;
6. inverse probe-flow maps for those macros;
7. one predictor quintet for every exact ZBLS algorithm.

The complete standard Speffz generator cycles are tabulated in [section A](#).

5.2 Trigger-level example

Under the standard Speffz convention, the sexy move

$$A = R \ U \ R' \ U'$$

has corner and edge cycles

$$\begin{aligned}\tau_A^C &= (A \ N \ R \ Q \ E \ B)(C \ P \ M \ V \ J \ K), \\ \tau_A^E &= (A \ J \ B)(M \ Q \ P).\end{aligned}$$

Therefore

$$\text{SBTDKA} \xrightarrow{RUR'U'} \text{SATDCJ}.$$

The six transformations are not six separate pieces of reasoning in the mature system. The target is one learned visual transformation of the entire word or constellation.

5.3 Algorithm card

Every exact macro A should have a machine-generated record

$$\text{Card}(A) = \left(\begin{array}{l} \text{moves, } au_A^C, \tau_A^E, \\ P_A^{\text{EO}}, b_A^{\text{EO}}, \\ T_A^{-1, \text{probe}}, \\ \Delta u, \Delta y, \Delta d, \\ \text{preservation masks and cognitive costs} \end{array} \right).$$

For an edge-orientation vector f , the macro acts affinely over $\mathbb{F}_2$:

$$f' = P_A^{\text{EO}} f \oplus b_A^{\text{EO}}.$$

An analogous affine action exists for corner twists over $\mathbb{Z}_3$.

5.4 Inspection-aware cost function

The shortest active-pair algorithm need not be the best inspection algorithm. A candidate can be scored by

$$\begin{aligned} J(A) = & w_m \text{Cost}_{\text{motor}}(A) + w_p \text{Cost}_{\text{pair}}(A, Q) \\ & + w_e \|b_A^{\text{EO}}\|_0 + w_q \text{Cost}_{\text{probe}}(A^{-1}Q) \\ & + w_r \text{Cost}_{\text{risk}}(A). \end{aligned}$$

The terms can include execution ergonomics, number of passive stickers moved, next-pair quality, EO disturbance, dispersion of future probes, rotation cost, and execution-error risk.

Design principle

The inspection-optimal algset is not necessarily the TPS-optimal algset. A one-move-longer algorithm may be globally superior if it leaves passive pairs coherent, preserves EO, localizes future probes, or yields a much easier successor state.

6 The Cross-Plus-Three Forward Compiler

6.1 State machine

At macro boundary k , define

$$S_k = (\Phi, m_k, \delta_k, Q_k, \mathcal{P}_k),$$

where

- Φ is the chosen color-neutral frame;
- $m_k \in \{0, 1\}^4$ is the solved-slot mask;
- $\delta_k \in \mathbb{Z}_4$ is the persistent pseudo-D phase;
- Q_k is the ordered ring of unsolved exact pair codes;
- $\mathcal{P}_k = [C, A_1, \dots, A_k]$ is the compiled macro stack.

The forward transition is

$$S_{k+1} = \mathcal{T}_{A_{k+1}}(S_k).$$

All passive pair codes are updated by the exact map of A_{k+1} ; the solved active pair is then deleted.

6.2 Destructive compilation

The pair code is a working register, not the final memo. Once the exact macro for a pair is selected, that pair's letters are discarded and replaced by a motor chunk. Thus inspection performs

$$\text{piece state} \longrightarrow \text{algorithm chunk}$$

three times.

For full ZB, all four pairs must initially be tracked because the final pair state is needed for ZBLS. The live pair-letter load is

$$8 \longrightarrow 6 \longrightarrow 4 \longrightarrow 2.$$

The exact number of macro-level letter transports is

$$8 + 6 + 4 + 2 = 20 :$$

eight through the cross, six through A_1 , four through A_2 , and two through A_3 .

6.3 Forward compiler procedure

Listing 1: Forward compiler for the cross and first three F2L pairs.

```

INPUT: untouched scramble x0

1. Choose frame  $\Phi = (D, F)$ .
2. Acquire  $Q = [q_0, q_1, q_2, q_3]$  in exact Speffz coordinates.
3. Choose an exact cross  $C$  using cross length and payload cost.
4. Apply  $T_C$  to all four pair codes.

5. For  $j = 1, 2, 3$ :
    a. Enumerate legal active-pair and target-slot choices.
    b. Normalize each exact pair state through safe setup/extraction.
    c. Retrieve candidate exact macros  $A$ .
    d. Score  $A$  for motor cost, passive-state quality, E0 behavior,
       probe behavior, and risk.
    e. Commit to one  $A_j$ .
    f. Apply  $T_{\{A_j\}}$  to every passive pair.
    g. Delete the solved pair; update mask and D phase.

OUTPUT:  $[C, A_1, A_2, A_3]$  and exact remaining-pair state  $q_{\text{star}}$ .

```

6.4 Color-neutral cross selection

A full exact search across all 24 frames is unlikely to fit inspection. Use a hierarchical policy:

1. perform a cheap scan over the six possible cross colors;
2. score cross length, first-pair accessibility, and pair-payload disturbance;
3. commit to one cross color;
4. choose the front direction to favor the best first pair and preferred hand;
5. only then acquire and compile the exact four-pair constellation in that frame.

A useful cross score is

$$J_C = \text{extmovecost} + \lambda \text{pair disturbance} + \mu \text{future probe-flow complexity}.$$

A six-move cross can be superior to a five-move cross if its transition map is much simpler.

7 Residual-State Mathematics for Full ZB

7.1 Paired corner and edge sites

After the cross and k F2L pairs are solved, let

$$n = 8 - k.$$

There are n remaining corners and n remaining non-cross edges. Pair the corner and edge sites by a fixed convention. One convenient initial pairing is

$$\begin{aligned} UFR &\leftrightarrow UF, & URB &\leftrightarrow UR, & UBL &\leftrightarrow UB, & ULF &\leftrightarrow UL, \\ DFR &\leftrightarrow FR, & DRB &\leftrightarrow RB, & DBL &\leftrightarrow BL, & DLF &\leftrightarrow LF. \end{aligned}$$

After solved pairs are removed, relabel the remaining paired sites by $0, \dots, n-1$.

Let

- $C \in S_n$ map each remaining corner site to the corner identity occupying it;
- $E \in S_n$ map each remaining edge site to the paired edge identity occupying it;
- $t \in \mathbb{Z}_3^{n-1}$ be the independent corner twists;
- $f \in \mathbb{F}_2^{n-1}$ be the independent edge flips.

7.2 Relative corner-edge permutation

Define

$$\rho = C^{-1} \circ E.$$

Operationally, $\rho(s)$ answers:

At which corner site is the corner whose identity is paired with the edge currently at edge site s ?

Proposition 7.1. *For every legal residual cube state, $\rho \in A_n$.*

Proof. A legal cube state has corner and edge permutations of equal parity. Fixed solved pieces do not alter this equality, so $\text{sgn}(C) = \text{sgn}(E)$. Therefore

$$\text{sgn}(\rho) = \text{sgn}(C^{-1}) \text{sgn}(E) = \text{sgn}(C) \text{sgn}(E) = +1.$$

Hence ρ is even and belongs to A_n . □

Because $E = C \circ \rho$, the exact remaining state can be coordinatized as

$$x_n \longleftrightarrow (C, t, \rho, f) \in S_n \times \mathbb{Z}_3^{n-1} \times A_n \times \mathbb{F}_2^{n-1}.$$

This is a coordinate factorization, not a statement that the cube group is a direct product.

7.3 Residual-state counts

The exact number of residual states is

$$N_n = \frac{(n!)^2 3^{n-1} 2^{n-1}}{2}.$$

Table 3: Exact residual state size at major solve boundaries.

Boundary	n	Exact residual states	Information
Cross solved	8	227,546,313,523,200	47.69 bits
Cross + 1 pair	7	592,568,524,800	39.11 bits
Cross + 2 pairs	6	2,015,539,200	30.91 bits
Cross + 3 pairs	5	9,331,200	23.15 bits
ZBLS complete, EO solved	4	7,776	12.92 bits

In particular,

$$9,331,200 = \underbrace{5!}_C \underbrace{3^4}_t \underbrace{\frac{5!}{2}}_{\rho \in A_5} \underbrace{2^4}_f,$$

and after ZBLS,

$$7,776 = \underbrace{4!}_C \underbrace{3^3}_t \underbrace{\frac{4!}{2}}_{\rho \in A_4}.$$

7.4 Contraction under solved pairs

Let a macro induce position permutations a_C and a_E on the remaining paired corner and edge sites. With the site-to-identity convention,

$$C' = C \circ a_C^{-1}, \quad E' = E \circ a_E^{-1}.$$

Therefore

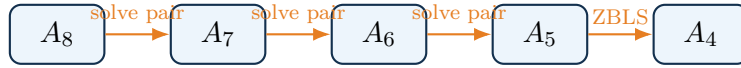
$$\boxed{\rho' = a_C \circ \rho \circ a_E^{-1}.$$

When the macro solves a pair at a common paired site s , delete that site and its solved identity. Denoting the induced deletion-and-relabeling map by D_s ,

$$\rho_{n-1} = D_s(a_C \circ \rho_n \circ a_E^{-1}).$$

The relative payload contracts through

$$A_8 \longrightarrow A_7 \longrightarrow A_6 \longrightarrow A_5 \longrightarrow A_4.$$



6-symbol code 5-symbol code 4-symbol code 3-symbol code 2-symbol code

Figure 2: Contraction of the relative corner-edge permutation as pairs are solved.

7.5 Minimal image code for A_n

Proposition 7.2. *An even permutation $\rho \in A_n$ is uniquely determined by the images*

$$R_n = (\rho(0), \rho(1), \dots, \rho(n-3)).$$

Proof. The displayed images are distinct. There are

$$n(n-1) \cdots 3 = \frac{n!}{2} = |A_n|$$

possible ordered injective lists. After the first $n-2$ images are fixed, two domain elements and two codomain values remain. The two possible completions have opposite parity, so exactly one is even. $\square$

Hence the exact relative-pairing payload shrinks as

$$6 \longrightarrow 5 \longrightarrow 4 \longrightarrow 3 \longrightarrow 2$$

symbols from A_8 to A_4 .

7.6 Why full forward tracking is not the preferred human method

The coordinate state (C, t, ρ, f) is mathematically elegant and sufficient. It also demands early acquisition of substantial last-layer identity and orientation information. A serial human implementation would have to carry that payload through every F2L macro.

The preferred competitive architecture therefore uses the exact forward model only for F2L pair state and delays the LL payload until it becomes live. This motivates the adjoint method developed next.

8 The Adjoint Full-ZB Architecture

8.1 State liveness

ZB needs two different edge data types at different times:

$$\boxed{\text{edge orientation before ZBLS} \oplus \text{edge permutation relative to corners after ZBLS.}}$$

At ZBLS selection, edge identities are unnecessary. At ZBLL selection, the old EO pattern is dead because ZBLS has set all LL edge orientations to zero.

This suggests the compiler principle:

F2L pair state	moves forward,
EO recognition questions	move backward when ZBLS becomes live,
ZBLL recognition questions	move backward after the exact ZBLS is known.

8.2 Probe pullback identity

Let x be a cube state and let G be an exact move sequence. Let $\chi_x(s)$ denote the sticker or color occupying oriented sticker position s in state x . If G moves sticker positions forward, then

$$\boxed{\chi_{Gx}(s) = \chi_x(G^{-1}s).}$$

A future observation at s can therefore be made on the untouched scramble by inspecting the inverse image $G^{-1}s$.

More generally, if h is any recognition query, define the pullback

$$G^*h = h \circ G.$$

Then

$$h(Gx) = (G^*h)(x).$$

This is the precise sense in which the method is “adjoint”: it transports output questions backward rather than transporting the entire state forward.

8.3 Prefix notation

Let the cross-plus-three prefix be

$$G_3 = A_3 \circ A_2 \circ A_1 \circ C.$$

Then

$$G_3^{-1} = C^{-1} \circ A_1^{-1} \circ A_2^{-1} \circ A_3^{-1}.$$

Operationally, a future probe is pulled to the initial scramble by applying

$$A_3^{-1}, \quad A_2^{-1}, \quad A_1^{-1}, \quad C^{-1}$$

in that order to the probe location.

If Z is the selected ZBLS macro, the prefix through ZBLS is

$$G_4 = Z \circ G_3,$$

and a final-LL probe is pulled back by

$$G_4^{-1} = G_3^{-1} \circ Z^{-1}.$$

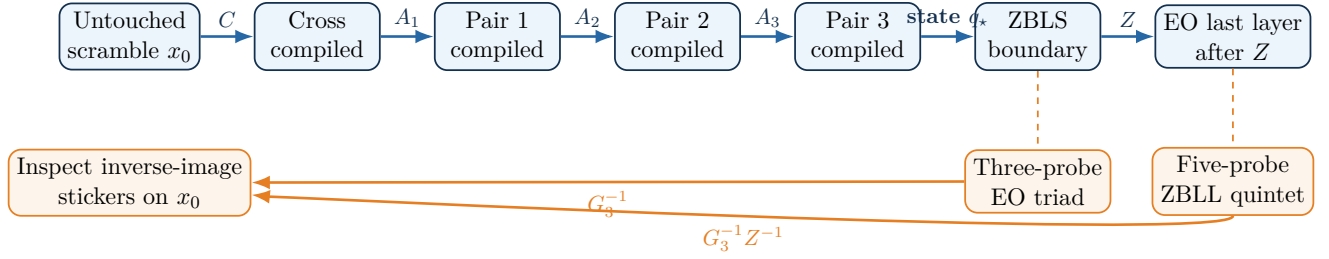


Figure 3: Forward F2L compilation and backward recognition-probe transport.

9 The Three-Probe EO Sensor for ZBLS

9.1 Five unsolved edges at the ZBLS boundary

After the cross and three F2L pairs are solved, five edge cubies remain unsolved:

- the final F2L edge;
- the four last-layer edge identities.

They occupy the four U -layer edge positions and the one open middle-layer slot. Let the final-pair edge orientation be $f_\star \in \mathbb{F}_2$, known from the exact remaining-pair code $q_\star$. Let the other four edge orientation bits be f_0, f_1, f_2, f_3 .

The global edge-orientation constraint gives

$$f_\star \oplus f_0 \oplus f_1 \oplus f_2 \oplus f_3 = 0.$$

Proposition 9.1. *Given $q_\star$, three binary edge-orientation probes determine the complete EO context required for ZBLS.*

Proof. The exact pair code determines $f_\star$. Probe any three of the four remaining unsolved edge positions, obtaining f_a, f_b, f_c . The fourth bit is

$$f_d = f_\star \oplus f_a \oplus f_b \oplus f_c.$$

Thus all five edge orientations are determined. $\square$

9.2 One-sticker orientation test

At each edge position, fix a primary reference facelet. Under the standard orientation convention:

- for an edge containing U or D , orientation zero means its U/D sticker lies on a U/D face;
- for a middle-layer edge, orientation zero means its designated F/B sticker lies on an F/B face.

Therefore one observed sticker color class at the selected facelet determines the edge's orientation bit.

The canonical EO sensor is a three-position glyph

$$\mathcal{E}(q_\star) = [e_a, e_b, e_c],$$

chosen from the four non-active unsolved edge positions. The omitted fourth position is reconstructed by parity. During early training, all four may be inspected, with the fourth serving as a checksum.

9.3 Adjoint EO query

Once q_* is known, form the future EO triad and pull it backward:

$$\mathcal{E}_0 = G_3^{-1} \mathcal{E}(q_*).$$

The solver applies the inverse flow maps of A_3, A_2, A_1, C to the three-point glyph, then observes the resulting three sticker positions on the untouched scramble.

The ZBLS case key is then

$$\kappa_{\text{ZBLS}} = \text{Canon}(q_*, f_0, f_1, f_2, f_3; u, y, \delta).$$

This selects one exact ZBLS card Z .

9.4 Why identities are not needed at this gate

ZBLS selection depends on the final-pair state and the orientation pattern of the five unsolved edges. It does not require the permutation identities of the four LL edges. Delaying those identities is the main state-liveness saving of the architecture.

The standard ZBLS family is conventionally described as 302 cases, or 125 algorithms when inverses and mirrors are grouped as equivalent [5]. The architecture is independent of the exact counting convention: each *executed sequence* receives its own transition and predictor card.

10 The Five-Probe Predictor for ZBLL

10.1 The exact post-ZBLS state

After ZBLS, F2L is solved and all four LL edges are oriented. The exact remaining state has

$$\frac{4! 4! 3^3}{2} = 7,776$$

possibilities before quotienting by algorithm-recognition conventions and AUFs.

Standard ZBLL solves such an edge-oriented last layer in one algorithm. The commonly used algset count is 493 cases including PLL; recognition is organized by corner orientation/COLL information and one of 12 relative edge-cycle patterns [6].

10.2 Three oriented corners determine the full corner state

Choose three of the four future LL corner positions. At each selected position, observe the complete oriented corner cubie: its identity and which sticker occupies a designated reference facelet.

Three observations determine three identities and three twists. The fourth identity is the unobserved identity, and its twist is determined by

$$t_0 + t_1 + t_2 + t_3 = 0 \pmod{3}.$$

Therefore three oriented corner observations determine the full corner permutation and orientation.

10.3 Two oriented edges determine the full edge permutation

Choose two of the four future LL edge positions and observe the side sticker at each. Because EO is solved, the side sticker identifies the edge identity.

After two edge identities are observed, two identities and two positions remain. There are exactly two possible assignments. One is even relative to the known corner permutation and one is odd. Legal cube parity selects the unique assignment whose edge permutation parity matches the corner permutation parity.

Proposition 10.1. *At an edge-oriented last layer,*

$$\boxed{3 \text{ oriented corners} + 2 \text{ oriented edges}}$$

determine the exact last-layer state.

10.4 One Speffz token per corner frame

A future corner observation is a complete oriented frame, but only one anchored sticker position must be transported backward. Cube moves preserve the cyclic handedness of a corner. Once the inverse image of one designated corner facelet is known, the other two stickers belong to the same initial corner and their cyclic order is fixed.

Thus define the canonical final sensor

$$\boxed{Q_\star = c_0 c_1 c_2 \mid e_0 e_1,}$$

where each c_i is one anchored corner-frame position and each e_i is one edge side-sticker position. It is a five-token Speffz-style query, even though each corner token instructs the solver to inspect an entire corner cubie.

10.5 ZBLS predictor quintet

For each exact ZBLS algorithm Z , store its predictor quintet

$$\boxed{Q_Z = Z^{-1} Q_\star.}$$

After selecting Z , recall Q_Z as part of the same memory card and pull it through the already-compiled F2L prefix:

$$\boxed{Q_0 = G_3^{-1} Q_Z = G_3^{-1} Z^{-1} Q_\star.}$$

Inspect the three resulting initial corner frames and two initial edge stickers. These observations reveal the exact future ZBLL state without ever carrying the four LL corners and four LL edges forward through the F2L prefix.

Algorithm-variant dependence

Two ZBLS algorithms that solve the same nominal case may produce different future AUFs, corner states, or edge cycles. Their predictor quintets are therefore different. The predictor must be attached to the exact sequence, not merely to a ZBLS case name.

10.6 Sensor-basis variants

Any three of four corners and any two of four edges suffice. Therefore there are at least

$$\binom{4}{3} \binom{4}{2} = 24$$

simple five-probe bases.

For a total prefix G , define a visibility cost

$$V_G(B) = w_f \# \{\text{faces containing } G^{-1}B\} + w_r \text{ reorientation cost} + w_s \text{ spatial dispersion.}$$

A mature system may select

$$B^\star = \arg \min_{B \in \mathcal{B}} V_G(B).$$

The progression should be one fixed basis, then left/right variants, then a small set of basis variants stored with each ZBLS family. Evaluating all 24 online is probably too expensive until the operation itself becomes chunked.

11 The A_4 Tetrahedral Edge Chunk

11.1 Relative edge permutation after ZBLS

At the final LL, number sites by

$$\begin{aligned} 0 &: UFR \leftrightarrow UF, \\ 1 &: URB \leftrightarrow UR, \\ 2 &: UBL \leftrightarrow UB, \\ 3 &: ULF \leftrightarrow UL. \end{aligned}$$

Let $C(s)$ be the corner identity at corner site s and $E(s)$ the paired edge identity at edge site s . Then

$$\rho = C^{-1} \circ E \in A_4.$$

Once the corner frame C is known, an observed edge identity at site s immediately yields

$$\rho(s) = C^{-1}(E(s)).$$

11.2 Why there are 12 holistic edge chunks

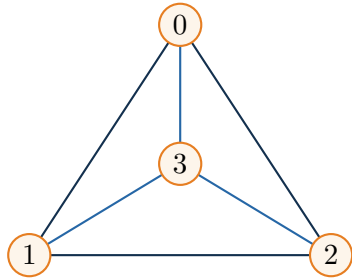
The group A_4 has

$$|A_4| = 12$$

elements:

$$1 \text{ identity} + 3 \text{ double transpositions} + 8 \text{ three-cycles}.$$

It is isomorphic to the rotational symmetry group of a tetrahedron. The final relative edge arrangement can therefore be learned as one of 12 tetrahedral rotations rather than as four independently tracked pieces.



$A_4 = 12$ orientation-preserving symmetries of the four labeled vertices

Figure 4: Tetrahedral mental model for the 12 final relative edge permutations.

11.3 Two-symbol code

An even permutation of four objects is uniquely determined by $\rho(0)$ and $\rho(1)$. Define

$$R_2 = \rho(0)\rho(1).$$

The values must be distinct, producing $4 \cdot 3 = 12$ codes. The remaining two images are the unused values in the unique order that gives even parity.

Table 4: Complete two-symbol code for A_4 . Cycle notation uses the action $i \mapsto \rho(i)$.

Code	Permutation	Code	Permutation
01	identity	10	(01)(23)
23	(02)(13)	32	(03)(12)
02	(123)	03	(132)
12	(012)	13	(013)
20	(021)	21	(023)
30	(031)	31	(032)

This supports a uniform ZBLL label of the form

$$\boxed{[\text{corner chunk}] - [\rho(0)\rho(1)]}.$$

For example, a personal code T3-20 can mean the third canonical T-corner configuration with relative edge token $20 = (021)$.

11.4 Color-neutral relational observables

Encode side colors by

$$F = 0, \quad R = 1, \quad B = 2, \quad L = 3 \pmod{4}.$$

For two side colors define the directed difference

$$d(c_1, c_2) = c_2 - c_1 \pmod{4}.$$

The values mean same, clockwise adjacent, opposite, and counterclockwise adjacent. A global y relabeling adds the same constant to both colors and leaves d invariant.

For sticker positions s, t , define

$$D_{s,t}(x) = d(\chi_x(s), \chi_x(t)).$$

Then

$$D_{s,t}(Gx) = D_{G^{-1}s, G^{-1}t}(x).$$

Thus future corner-edge relations can be read directly from pairs of initial stickers after pulling the relation query backward. Standard ZBLL recognition already uses local same/adjacent/opposite relationships around a recognized corner configuration [6]; the directed relation refines this into a fully color-neutral four-valued observable.

12 Complete Online Inspection Algorithm

12.1 Exact procedure

The complete theoretical procedure is:

1. Choose the cross color and oriented frame $\Phi = (D, F)$.
2. Acquire all four exact F2L pair codes $Q = [q_0, q_1, q_2, q_3]$.
3. Choose an exact cross C using motor and payload cost.
4. Apply T_C to all four pairs.
5. Choose A_1 ; transform the three passive pairs; delete the solved pair.

6. Choose A_2 ; transform the two passive pairs; delete the solved pair.
7. Choose A_3 ; transform the final passive pair; delete the solved third pair.
8. The remaining exact pair state is q_* .
9. Construct the canonical EO triad $\mathcal{E}(q_*)$.
10. Pull it backward through $A_3^{-1}, A_2^{-1}, A_1^{-1}, C^{-1}$ and inspect the three initial stickers.
11. Reconstruct the fourth EO bit by parity and select the exact ZBLS algorithm Z .
12. Recall Z 's predictor quintet $Q_Z = Z^{-1}Q_*$.
13. Pull Q_Z backward through $A_3^{-1}, A_2^{-1}, A_1^{-1}, C^{-1}$.
14. Inspect the three initial corner frames and two initial edge stickers.
15. Reconstruct the exact future corner state and the two-symbol A_4 edge token.
16. Select the exact ZBLL algorithm L , including AUF conventions.
17. Discard all symbolic probes and rehearse only $[C][A_1][A_2][A_3][Z][L]$.

12.2 Compiler pseudocode

Listing 2: Full-ZB inspection compiler.

```

INPUT: untouched scramble x0

Phi <- choose_frame(x0)
Q   <- acquire_four_pair_constellation(x0, Phi)
C   <- choose_cross(x0, Q, Phi)
Q   <- forward_pair_map(C, Q)
prefix <- [C]

for j in {1,2,3}:
    candidates <- legal_pair_macros(Q, solved_mask, D_phase)
    A_j <- argmin_A inspection_cost(A, Q, prefix)
    Q <- forward_map_passive_pairs(A_j, Q)
    Q <- remove_solved_pair(Q)
    update(solved_mask, D_phase, frame_phase)
    prefix.append(A_j)

q_star <- only_remaining_pair(Q)

E_future <- canonical_EO_triad(q_star)
E_initial <- pullback(E_future, inverse(prefix))
EO_bits <- inspect_orientation_bits(x0, E_initial)
EO_full <- parity_complete(EO_bits, orientation(q_star.edge))

Z <- exact_ZBLS_lookup(q_star, EO_full, phases)
Q_Z <- stored_predictor_quintet(Z) # equals Z^-1(Q_final)
Q_initial <- pullback(Q_Z, inverse(prefix))

corner_state <- inspect_three_corner_frames(x0, Q_initial.corners)
edge_obs <- inspect_two_edge_stickers(x0, Q_initial.edges)
edge_state <- parity_complete_edges(edge_obs, corner_state)
rho_code <- relative_A4_code(corner_state, edge_state)

```

```

L <- exact_ZBLL_lookup(corner_state, rho_code, phases_after_Z)

RETURN [C,A1,A2,A3,Z,L]

```

12.3 Core invariants

At every macro boundary, the following must hold:

1. every live pair code agrees with the physical state predicted by the compiled prefix;
2. the solved-slot mask and pseudo-D phase agree with the chosen algorithms;
3. every exact algorithm card matches the precise sequence that will be executed;
4. the EO triad omits exactly one of the four non-active unsolved edges;
5. the ZBLL quintet is defined in the final frame produced by the selected ZBLS card;
6. AUF, y frame, and D phase are represented exactly once, not both absolutely and as separate gauges.

13 Algorithm-Set Design for Predictive Inspection

13.1 ZBLS card

Each exact ZBLS sequence should store

$$\text{ZCard}(Z) = \left(\begin{array}{l} \text{canonical pair case and EO key,} \\ \text{exact motor sequence,} \\ Q_Z = Z^{-1}Q_\star, \\ \text{optional redundant check probe,} \\ \Delta u, \Delta y, \Delta d, \\ \text{motor and probe costs} \end{array} \right).$$

The motor sequence and predictor glyph should be memorized as one inseparable object.

13.2 Probe-overlap optimization

At the pre-ZBLS boundary, the EO triad and ZBLL predictor may overlap. Define

$$N_{\text{probe}}(Z) = \left| \mathcal{E}(q_\star) \cup Z^{-1}Q_\star \right|.$$

Because a prefix inverse is bijective, it preserves intersections and cardinality:

$$\left| G_3^{-1}\mathcal{E}(q_\star) \cup G_3^{-1}Z^{-1}Q_\star \right| = N_{\text{probe}}(Z).$$

Probe overlap can therefore be optimized offline for every ZBLS case, independently of the preceding cross and F2L prefix.

An inspection-aware ZBLS generator should optimize

motor cost + EO/quintet overlap + predictor locality + frame cleanliness + execution reliability.

13.3 Payload-transparent macros

A macro is cognitively favorable when paired corners and edges move nearly in lockstep. In the relative coordinate,

$$\rho' = a_C \rho a_E^{-1}.$$

If a_C and a_E have similar support and geometry, the relative pairing is less distorted and easier to chunk. An EO-transparent macro has

$$b_A^{\text{EO}} = 0.$$

Algorithms composed primarily of moves that preserve the chosen EO convention reduce the amount of orientation bookkeeping, although exact behavior must always be generated from the sequence.

13.4 Deterministic policy

Prediction requires exact reproducibility. During training, freeze:

- one exact algorithm per canonical case and slot family;
- one setup/extraction policy;
- one tie-breaking policy for pair order;
- one ZBLS predictor basis;
- one ZBLL labeling and AUF convention.

Improvising between equivalent algorithms invalidates the prelearned successor and predictor maps.

13.5 Superchunks

Once individual macro maps are fluent, frequently adjacent macro pairs can be stored as compositions:

$$T_{A_i A_j} = T_{A_j} \circ T_{A_i}, \quad (A_i A_j)^{-1} = A_i^{-1} A_j^{-1}.$$

A superchunk reduces two scene transformations to one. The useful set should be learned from empirical frequency rather than exhaustively; otherwise the combinatorial library becomes unmanageable.

14 Holistic Human Representations

14.1 The F2L constellation

The mature state should not feel like eight spoken letters. It should be a four-pair spatial constellation: four corner markers and four edge markers attached to a mental cube. A macro applies one learned flow field to the whole constellation.

Speffz remains the exact language for debugging and deliberate study. With sufficient overlearning, it should recede in competition in the same way that individual letters recede while reading a familiar word.

14.2 The EO triad

The three edge probes should be encoded as one rigid three-point glyph. Pulling it through an inverse macro should feel like moving a triangular object through a learned flow, not performing three independent table lookups.

14.3 The predictor quintet

The five-probe sensor has a natural shape:

$$\boxed{\text{corner triangle } \triangle \quad + \quad \text{edge bar } - - .}$$

The three corner frames form the triangle; the two edge probes form the bar. Each ZBLS case retrieves both its motor chunk and a case-specific triangle-plus-bar predecessor glyph.

14.4 The tetrahedral edge token

After the corner frame is decoded, the two observed edges produce one of 12 A_4 states. The target is immediate retrieval of a tetrahedral rotation or two-symbol token, not reconstruction of four separate edge trajectories.

14.5 Self-indexing motor chain

The final memo is

$$[C][A_1][A_2][A_3][Z][L].$$

It is not a flat sequence of turns. Executing C creates the physical cue for A_1 ; A_1 creates the cue for A_2 ; and so forth. The planned solve is therefore a self-indexing chain of motor states. This reduces working-memory burden during execution, provided every macro boundary has been compiled correctly.

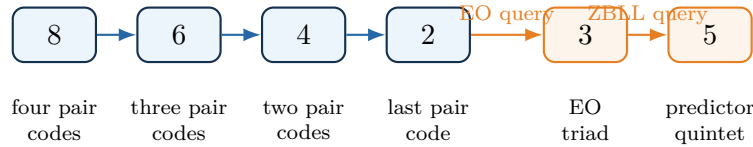


Figure 5: Live symbolic register schedule. The orange query registers are temporary and are discarded immediately after decoding.

15 Training Curriculum

The curriculum is organized by mastery gates rather than calendar duration. A later stage begins only when the previous stage is accurate under distraction and time pressure.

Table 5: Progressive training program.

Stage	Capability	Primary drills	Exit criterion
0	Dynamic frame and Speffz acquisition	Relabel random cube orientations into a chosen D/F frame; identify corner and edge letters without solving.	Near-perfect letter accuracy from all 24 frames.
1	Face-turn generators	Transform random corner and edge letters under U, R, F, D, L, B , then inverses and doubles.	No visual cube simulation required for single turns.
2	Trigger maps	Sexy, inverse sexy, sledge, hedge, standard insertions, and safe extractions as one-step word maps.	Six- and eight-letter words transformed as single chunks.

Stage	Capability	Primary drills	Exit criterion
3	Cross payload transport	Inspect four pairs, choose a fixed-color cross, and state the exact post-cross ring.	Reliable untimed reconstruction verified by simulator.
4	Fixed-policy cross + three	Use one frame, deterministic pair order, and fixed algorithms.	Exact remaining-pair prediction on random scrambles.
5	EO triad pullback	Given a known prefix and q_* , pull three orientation probes backward.	Three-bit prediction plus parity checksum is consistently correct.
6	Predictor quintets	Learn a small ZBLS subset with motor sequence and quintet as one card.	Exact future corner/edge observations for that subset.
7	A_4 decoder	Drill all 12 two-symbol codes and convert observed edges to relative permutations.	Instant code-to-case and case-to-code recall.
8	Full fixed-frame ZB compiler	Combine forward pair compilation, EO query, ZBLS, quintet query, and ZBLL.	Full planned solution correct untimed across a large test set.
9	Optimization	Add adaptive pair order, multiple sensor bases, pseudo-D states, superchunks, and inspection-aware alg choices.	Measurable latency reduction without accuracy loss.
10	Full color neutrality and pressure	Six cross colors, competition calls, hand-placement routine, and fallback policy.	Reproducible performance below the chosen internal time boundary.

15.1 Error taxonomy

Every failed trial should be classified before repetition:

- frame-selection error;
- initial visual-acquisition error;
- face-turn or macro-map error;
- wrong exact algorithm variant;
- solved-slot or D -phase error;
- EO reference-facelet error;
- inverse-order error during pullback;
- corner-frame observation error;
- parity-completion error;
- A_4 decode error;

- final motor-memory error.

Latency should never be optimized by hiding an unclassified accuracy failure.

15.2 Training order for ZBLS and ZBLL

Do not begin by attaching predictors to hundreds of unrelated sequences. Build families:

1. one last-slot pair shape;
2. all EO variants of that shape;
3. mirrors and inverses generated from the base cards;
4. neighboring shapes sharing triggers;
5. full ZBLS only after predictor transformations are automatic.

For ZBLL, impose the same 12-position A_4 order inside every corner subset. This converts ZBLL from hundreds of unrelated names into a product-like recognition system:

$$[\text{corner family and subset}] \times [A_4 \text{ edge token}].$$

16 Software Trainer and Verification System

16.1 Required macro record

Listing 3: Suggested machine record for an exact macro.

```
Macro:
  name
  moves
  corner_sticker_permutation[24]
  edge_sticker_permutation[24]
  corner_position_permutation[8]
  corner_orientation_delta[8]
  edge_position_permutation[12]
  edge_orientation_delta[12]
  inverse_probe_flow
  solved_slot_effect
  D_phase_delta
  U_phase_delta
  y_frame_delta
  motor_cost
  pair_distortion_cost
  probe_dispersion_cost
  risk_score
```

A ZBLS card additionally stores:

```
ZBLSCard:
  canonical_last_pair_case
  canonical_EO_pattern
  exact_macro_id
  predictor_quintet
  optional_check_probe
  final_U_phase
  final_y_frame
```

```
final_D_phase
compatible_sensor_bases
```

16.2 Drill modes

The trainer should implement at least:

- **forward_constellation**: $Q + A \mapsto Q'$;
- **inverse_triad**: EO glyph $+A \mapsto$ predecessor glyph;
- **inverse_quintet**: predictor glyph $+A \mapsto$ predecessor glyph;
- **A4_decode**: corner frame $+ \text{two edges} \mapsto \rho(0)\rho(1)$;
- **normalizer**: arbitrary $k = 0, 1, 2$ pair state $\mapsto$ safe extraction and core case;
- **full_compile**: scramble $\mapsto [C][A_1][A_2][A_3][Z][L]$.

16.3 Property-based correctness tests

The software must validate the theory automatically on random legal states:

1. **Generator correctness**: each stored face-turn map agrees with an independent facelet simulator.
2. **Composition correctness**: the map of a move string equals the composition of its generators.
3. **Pair-flow correctness**: every predicted Speffz pair code equals the simulator state.
4. **Pullback identity**: for random x, G, s ,

$$\chi_{Gx}(s) = \chi_x(G^{-1}s).$$

5. **EO triad sufficiency**: the three probes plus f_* reproduce all five unsolved-edge orientation bits.
6. **Quintet sufficiency**: the five observations reproduce the exact post-ZBLS LL state.
7. **A_4 parity**: every decoded relative permutation is even and matches direct simulation.
8. **End-to-end solve**: the compiled six-macro solution solves the original scramble exactly.

16.4 Metrics

Record separate latency and accuracy distributions for:

- frame choice;
- four-pair acquisition;
- cross transport;
- each F2L macro selection and passive update;
- EO triad pullback and reading;
- ZBLS lookup;
- quintet pullback and reading;
- A_4 decode;
- ZBLL lookup;

- final six-chunk rehearsal.

A total inspection time is diagnostically weak unless its components are known.

16.5 Algorithm search objective

Candidate algorithm generation should use a multiobjective Pareto search rather than minimizing move count alone. Candidate A dominates B only if it is no worse in motor cost, transition simplicity, EO behavior, probe locality, and reliability, and strictly better in at least one dimension.

The resulting personal algset is an interface between a cube solver and a cognitive compiler.

17 Inspection Timing and Fallback Architecture

17.1 Aspirational mature schedule

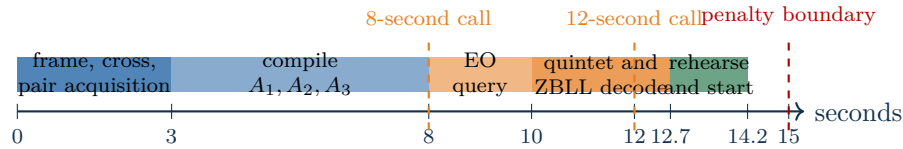


Figure 6: Aspirational mature schedule. The segmentation is a design target, not a demonstrated universal human result.

A reasonable target allocation is:

0–3 s	Choose frame and cross; acquire four-pair constellation.
3–8 s	Compile A_1, A_2, A_3 ; lock q_* .
8–10 s	Pull and read EO triad; select Z .
10–12.7 s	Pull and read predictor quintet; decode corner chunk and A_4 token; select L .
12.7–14.2 s	Rehearse six motor chunks, orient cube, place hands, and start.

17.2 Fallback ladder

Competition training should include explicit degraded modes:

1. If the three-pair prefix is not fixed by the 8-second call, stop deeper prediction and begin with the reliable planned prefix available.
2. If ZBLS is known but the quintet is incomplete near the 12-second call, retain $[C][A_1][A_2][A_3][Z]$ and recognize ZBL during execution.
3. If the EO query is uncertain, use a safe last-slot algorithm and ordinary LL recognition rather than executing a misclassified ZBLS.
4. Never sacrifice a legal start for completion of the theoretical plan.

The terminal research goal remains universal full compilation; the fallback ladder prevents training ambition from producing avoidable competition penalties.

18 Feasibility, Limits, and Research Questions

18.1 Information-theoretic possibility

The complete 3x3 state space has

$$8! 3^7 12! 2^{10} = 43,252,003,274,489,856,000$$

states, corresponding to

$$\log_2 N \approx 65.23 \text{ bits.}$$

The complete solution is therefore a finite deterministic function of roughly 65 bits of cube state. There is no information-theoretic impossibility in selecting a full solution during inspection.

18.2 Why information theory is not the bottleneck

The practical difficulty lies in:

- visual acquisition across an opaque three-dimensional object;
- dynamic color-neutral relabeling;
- exact macro selection under severe time pressure;
- parallel transformation of spatial constellations;
- retaining the prefix while performing inverse probe flow;
- mapping a final observation to one of hundreds of motor chunks;
- robustness to a single execution error, which invalidates the predicted suffix.

An explicit serial implementation remains too slow. The architecture succeeds only if letter transformations collapse into holistic scene transformations and algorithm cards become direct perception-action associations.

18.3 Symbolic-operation budget

A serial full-ZB version would require approximately:

- 20 pair-letter transports in the forward prefix;
- 12 individual probe transports for a three-point EO glyph through four macros;
- 20 individual probe transports for a five-point quintet through four prefix macros, after Z^{-1} is precompiled into the card.

That is approximately 52 individual token lookups, too many for a conscious 15-second procedure.

The holistic target replaces these with approximately:

$$4 \text{ forward constellation scenes} + 4 \text{ inverse EO-glyph scenes} + 4 \text{ inverse quintet scenes,}$$

plus recognition and lookup. Common superchunks can reduce this count further.

18.4 Central empirical hypotheses

The research program should test:

1. whether eight exact pair-sticker positions can be acquired as one spatial packet faster than as eight verbal letters;

2. whether inverse glyph transport becomes as fast as forward trigger recognition after overlearning;
3. whether predictor-locality-optimized algorithms materially reduce inspection time;
4. whether two-symbol A_4 organization improves ZBLL recognition and retention;
5. whether a fixed deterministic algset outperforms ergonomic improvisation for full prediction;
6. whether adaptive sensor-basis selection saves more visual time than it costs cognitively;
7. whether full color neutrality remains beneficial after probe-transport costs are included.

18.5 Honest conclusion on attainability

Current assessment

- Color-neutral cross plus three pairs is mathematically clean and a plausible long-term trainable capability.
- Cross plus three plus exact ZBLS prediction is substantially harder, but the three-probe EO sensor gives it a credible architecture.
- Universal full-ZB planning in legal inspection is mathematically possible and not ruled out by state size. Human attainability remains speculative. It requires parallelized visual-spatial transformations far beyond simply memorizing additional algorithms.

19 Implementation Roadmap

19.1 Phase I: exact simulator and cards

Build a reference simulator that parses Singmaster notation, generates 24-corner-sticker and 24-edge-sticker permutations, canonicalizes cases, and exports algorithm cards. Verify every map by property-based tests before human training begins.

19.2 Phase II: fixed-frame proof of concept

Use one cross color, one front, one target last slot, and a small deterministic F2L/ZBLS subset. Demonstrate exact forward pair prediction, EO triad recovery, and quintet recovery without a time limit.

19.3 Phase III: complete fixed-frame system

Expand to all pair states, safe extraction normalizers, full ZBLS, and full ZBLL while retaining a single frame and frozen policy. Measure accuracy before speed.

19.4 Phase IV: cognitive optimization

Search for inspection-optimal algorithms, predictor overlap, localized sensor bases, frequent superchunks, and compact visual glyphs. Compare them experimentally against move-count-optimal algorithms.

19.5 Phase V: adaptive and color-neutral system

Add pair-order beam search, all four slots, persistent pseudo-D phase, six cross colors, and competition timing. Use a cheap first-stage frame heuristic to avoid exact planning in all 24 frames.

19.6 Phase VI: universal test

For large random-scramble sets, require the software to certify the compiled solution and classify every human failure. The universal claim should be made only if the method succeeds across unbiased random scrambles, not hand-selected favorable cases.

20 Condensed Reference Architecture

Forward: $Q_0 \xrightarrow{C} Q_1 \xrightarrow{A_1} Q_2 \xrightarrow{A_2} Q_3 \xrightarrow{A_3} q_\star$

EO query: $\mathcal{E}_0 = G_3^{-1} \mathcal{E}(q_\star) \implies Z$

ZBL query: $\mathcal{Q}_0 = G_3^{-1} Z^{-1} \mathcal{Q}_\star \implies (C, t, \rho_4)$

Edge chunk: $\rho_4 \in A_4, \quad R_2 = \rho(0)\rho(1)$

Final memo: $[C][A_1][A_2][A_3][Z][L]$

Central conceptual result

The scalable route to full-ZB inspection is not a larger blindfold memo. It is a compiler with exact transition operators, live-variable scheduling, and adjoint recognition probes:

four F2L pairs forward $\longrightarrow$ three EO bits backward
 $\longrightarrow$ five final-state probes backward $\longrightarrow$ one A_4 edge token

A Standard Speffz Generator Maps

The following convention is used: a cycle $(A B C D)$ means that the sticker currently at A moves to B , the sticker at B moves to C , and so on. The maps assume standard Singmaster clockwise turns viewed directly at the turned face and the standard $ULFRBD$ Speffz scheme.

Table 6: Clockwise face-turn maps.

Move	Corner-sticker cycles	Edge-sticker cycles
U	$(A B C D)(E Q M I)(F R N J)$	$(A B C D)(E Q M I)$
R	$(B T V J)(C Q W K)(M N O P)$	$(B T V J)(M N O P)$
F	$(C P U F)(D M V G)(I J K L)$	$(C P U F)(I J K L)$
D	$(G K O S)(H L P T)(U V W X)$	$(G K O S)(U V W X)$
L	$(A I U S)(D L X R)(E F G H)$	$(D L X R)(E F G H)$
B	$(A H W N)(B E X O)(Q R S T)$	$(A H W N)(Q R S T)$

For an inverse move, traverse each cycle backward. For a double move, advance two positions in every four-cycle.

A.1 Whole-cube frame maps

Under the standard cubing convention that x rotates the whole cube like an R move and y like a U move, the corner and edge alphabets transform identically:

$$\begin{aligned} x &= (A S U I)(B T V J)(C Q W K) \\ &\quad (D R X L)(E H G F)(M N O P), \\ y &= (A B C D)(E Q M I)(F R N J) \\ &\quad (G S O K)(H T P L)(U X W V). \end{aligned}$$

The y map generates all four slot translations. The pair-state compiler should use these maps to generate slot variants instead of memorizing separate tables whenever the move sequences are true conjugates.

B Detailed Derivations

B.1 Full cube state count

The eight corners have $8!$ permutations and 3^7 independent orientations. The twelve edges have $12!$ permutations, 2^{11} orientation assignments, and a parity constraint linking their permutation parity to the corners. Therefore

$$8! 3^7 \frac{12! 2^{11}}{2} = 8! 3^7 12! 2^{10}.$$

B.2 Post-three-pair suffix count

With the cross and three F2L pairs solved, five corners and five edges remain. Their permutations have $(5!)^2/2$ legal parity-matched possibilities, their orientations have 3^4 and 2^4 possibilities, so

$$\frac{(5!)^2}{2} 3^4 2^4 = 9,331,200.$$

A direct table from this exact state to a ZBLS/ZBLL suffix would therefore contain millions of entries. The adjoint architecture factors the decision into a last-pair/EO gate followed by a final-LL gate.

B.3 Post-ZBLS state and the 12 edge relations

With F2L solved and EO complete, four corners and four edges remain. The count is

$$\frac{(4!)^2}{2} 3^3 = 7,776.$$

Factoring into $C \in S_4$, $t \in \mathbb{Z}_3^3$, and $\rho \in A_4$ gives

$$24 \cdot 27 \cdot 12 = 7,776.$$

The 12-element A_4 factor is precisely the final edge relationship once the corner frame is known.

C Quick Reference Sheet

Exact pair state

Track the D sticker of each corner and the leading side sticker of each F2L edge.

$$q_i = (\gamma_i, \epsilon_i), \quad Q = [q_0, q_1, q_2, q_3].$$

Pair update

$$T_A(q_i) = (\tau_A^C(\gamma_i), \tau_A^E(\epsilon_i)).$$

Forward prefix

$$G_3 = A_3 \circ A_2 \circ A_1 \circ C.$$

EO query

$$\mathcal{E}_0 = G_3^{-1} \mathcal{E}(q_\star).$$

Three bits plus $f_\star$ determine all five unsolved-edge orientations.

ZBLL query

$$\mathcal{Q}_Z = Z^{-1} \mathcal{Q}_\star, \quad \mathcal{Q}_0 = G_3^{-1} \mathcal{Q}_Z.$$

Observe three corner frames and two edge side stickers.

Relative edge token

$$\rho = C^{-1} E \in A_4, \quad R_2 = \rho(0) \rho(1).$$

Live registers

$$8 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 5.$$

Final motor memo

$$[C][A_1][A_2][A_3][Z][L].$$

Clean interfaces

$$\delta = 0 \text{ before standard ZBLS,}$$

and all AUF/frame phases must be represented exactly once.

Cycle convention

$(A B C D)$ means the sticker at A moves to B .

Competition calls

8 seconds and 12 seconds; start strictly before 15.00 seconds [1].

References

- [1] World Cube Association. *WCA Regulations*, Article A, Regulations A3 and A4. Effective April 1, 2026; accessed September 1, 2026. <https://www.worldcubeassociation.org/regulations/>
- [2] Speedsolving.com Wiki. *Speffz*. Accessed September 1, 2026. <https://www.speedsolving.com/wiki/index.php/Speffz>
- [3] Stefan Pochmann. *Single Algorithm 3x3 Blindsolving*. Discussion of oriented edge and corner sticker distinctions. Accessed September 1, 2026. https://www.stefan-pochmann.info/spocc/blindsolving/3x3_single_alg/old_index.php
- [4] Herbert Kociemba. *The Cubie Level* and *The Coordinate Level*. Accessed September 1, 2026. <https://kociemba.org/math/cubielevel.htm> <https://kociemba.org/math/coordlevel.htm>
- [5] Speedsolving.com Wiki. *ZBLS*. Accessed September 1, 2026. <https://www.speedsolving.com/wiki/index.php?title=ZBLS>
- [6] Speedsolving.com Wiki. *ZBLL*. Accessed September 1, 2026. <https://www.speedsolving.com/wiki/index.php/ZBLL>

D What “full mastery” would mean

Under the strongest interpretation, full mastery of the Adjoint ZB Compiler would mean that, for essentially any legal scramble, you could use inspection to produce a complete executable program

$$\mathcal{P}(x_0) = [C, A_1, A_2, A_3, Z, L],$$

where

- C solves the selected color-neutral cross,
- A_1, A_2, A_3 solve the first three F2L pairs,
- Z is the exact ZBLS algorithm for the remaining pair and EO state,
- L is the exact ZBLL algorithm, including the necessary AUF,

and

$$LZA_3A_2A_1C(x_0) = e.$$

You would begin the timer already knowing every major motor chunk of the solve. That would have implications far beyond simply “having better lookahead.”

D.1 The solve would change from reactive to compiled

Ordinary speed-solving is fundamentally reactive:

$$\text{execute} \rightarrow \text{observe} \rightarrow \text{recognize} \rightarrow \text{choose} \rightarrow \text{execute}.$$

The Adjoint ZB Compiler changes this to

$$\text{inspect} \rightarrow \text{compile} \rightarrow \text{execute}.$$

During the timed portion, you would no longer be continuously asking:

- Where is the next pair?
- Which pair should I solve?
- What F2L case is this?
- What ZBLS case did I produce?
- Which ZBLL is this?
- What AUF do I need?

Those questions would already have been answered.

A conventional solve can be decomposed approximately as

$$T_{\text{solve}} = T_{\text{motor}} + T_{\text{recognition}} + T_{\text{search}} + T_{\text{decision}} + T_{\text{transition}} + T_{\text{recovery}}.$$

With full compiler mastery,

$$T_{\text{AZC}} \approx T_{\text{motor}} + T_{\text{transition}} + T_{\text{verification}} + T_{\text{recovery}}.$$

The recognition, search, and decision terms have been moved into inspection, which does not count toward the official solve time provided that the solve starts within the inspection limit.

That is the central competitive advantage: **untimed cognition is exchanged for timed motor performance.**

D.2 Lookahead would almost disappear from the timed solve

Normal F2L lookahead is the skill of locating and identifying the next pair while executing the current pair. Under the compiler architecture, there is nothing left to search for. The next pair, algorithm, and resulting state are already known.

The role of vision during execution would change from **planning** to **verification**.

Instead of visually searching the cube, you would check a few predicted landmarks:

$C \Rightarrow$ expected first-pair configuration,
 $A_1 \Rightarrow$ expected second-pair configuration,
 $A_2 \Rightarrow$ expected third-pair configuration,
 $A_3 \Rightarrow$ expected ZBLS configuration,
 $Z \Rightarrow$ expected ZBLL configuration.

The solve would therefore be mostly open-loop but sparsely corrected by closed-loop feedback:

precompiled feedforward execution + checkpoint verification.

You could potentially execute large portions of the solve without visually depending on the cube. Closing your eyes would not be competitively optimal because visual feedback catches finger errors, but the solve would be conceptually close to a one-look blind execution.

D.3 Solve-time variance could fall dramatically

One of the largest benefits might not be the absolute fastest single. It could be the reduction in bad solves.

Ordinary solve variance comes from:

- Losing a pair.
- Failing to predict where a pair will emerge.
- Choosing a poor pair order.
- Encountering an awkward rotation.
- Pausing before last-slot recognition.
- Misrecognizing ZBLL.
- Failing to see AUF immediately.
- Becoming surprised by a case.

Correct full compilation removes nearly all of those sources.

The timed solve would become more dependent on relatively stable factors:

- Finger speed.
- Regrips.
- Lockups.
- Algorithm ergonomics.
- Physical execution consistency.

Thus the distribution of solve times could become narrower. Long pauses and catastrophic recognition failures would largely disappear—provided the inspection compilation itself was correct.

The tradeoff is that compiler errors are highly correlated. One incorrect state transformation can corrupt everything after it:

$$\hat{A}_1 \neq A_1 \implies \hat{x}_2 \neq x_2 \implies \hat{A}_2, \hat{A}_3, \hat{Z}, \hat{L} \text{ may all be wrong.}$$

So the performance distribution could become bimodal:

- Extremely smooth, fully compiled solves.
- Severe failures caused by one inspection error.

This makes accuracy more important than raw compilation speed.

A simplified expected-time comparison is

$$\mathbb{E}[T_{\text{AZC}}] = (1 - p_e)T_c + p_e T_r,$$

where

- p_e is the probability of a compilation or retention error,
- T_c is the clean compiled execution time,
- T_r is the resulting recovery time or penalty-equivalent cost.

Against a conventional average T_b , compilation is beneficial when

$$p_e < \frac{T_b - T_c}{T_r - T_c}.$$

Because T_r can be very large, full competitive mastery would require an extremely low error rate.

D.4 F2L would become a global optimization problem

Ordinary F2L decisions are often locally greedy:

Which visible pair is easiest right now?

Full compilation allows a more powerful question:

Which complete path from this scramble produces the lowest-cost total solve?

The planner could optimize

$$\pi^* = \arg \min_{\pi} [T_{\text{execution}}(\pi) + \lambda C_{\text{inspection}}(\pi) + \mu R_{\text{error}}(\pi)],$$

where

$$\pi = [C, A_1, A_2, A_3, Z, L].$$

This would allow decisions such as:

- Taking a slightly longer cross because it preserves three favorable pairs.
- Solving a less obvious first pair because it produces two connected pairs.
- Choosing a five-move insertion instead of a four-move insertion because it leaves the next pair unchanged.
- Selecting an F2L algorithm that produces an easier ZBLS.
- Selecting between two ZBLS algorithms based on the resulting ZBLL.
- Deliberately producing a preferred ZBLL subset.
- Favoring an algorithm whose predictor probes remain visually clustered.
- Choosing a solution with fewer regrips even if its move count is higher.

This is essentially finite-horizon optimal control:

$$x_{k+1} = T_{A_k}(x_k),$$

with the objective

$$\min_{A_0, \dots, A_5} \sum_{k=0}^5 c(A_k)$$

subject to

$$x_6 = e.$$

The method would therefore elevate F2L from a sequence of local pair solutions into a globally coordinated trajectory.

Full mastery would not automatically imply optimal solutions. There are too many possible branches to search exhaustively during inspection. But it would make **global optimization possible in principle**, using learned heuristics, beam search, or a memorized policy.

D.5 Algorithm quality would acquire a second dimension

Traditional speedcubing algorithm selection emphasizes:

- Move count.
- Fingertricks.
- Regrips.
- Rotations.
- TPS.

- Recognition.

The Adjoint ZB Compiler adds a new category:

information-transform complexity.

Each algorithm is simultaneously:

1. A motor sequence.
2. A permutation operator on all passive pieces.
3. An EO transformation.
4. An inverse flow field for future probes.
5. A transformation of frame, AUF, and pseudo- D phase.

Two algorithms that solve the same F2L case might have very different cognitive costs. One could define an inspection-aware algorithm objective such as

$$J(A) = w_m c_{\text{motor}}(A) + w_p c_{\text{passive pair}}(A) + w_e c_{\text{EO}}(A) + w_q c_{\text{probe}}(A) + w_r c_{\text{retention}}(A).$$

This could create a completely new class of algsets:

D.5.1 Payload-transparent algorithms

Algorithms that move passive corners and edges in nearly corresponding ways, preserving relational pair structure.

D.5.2 EO-transparent algorithms

Algorithms that leave the passive EO field unchanged or transform it in a very simple way.

D.5.3 Probe-local algorithms

Algorithms whose inverse maps keep the EO triad and ZBLL predictor quintet clustered on a small number of visible faces.

D.5.4 Frame-clean algorithms

Algorithms that end with predictable AUF, slot, and cube-frame conventions.

D.5.5 Compiler-friendly algorithms

Algorithms that may be marginally slower in isolation but reduce the total cognitive burden enough to improve complete solves.

That would produce a Pareto frontier:

motor efficiency versus inspection transparency.

The best full-solve algorithm might no longer be the fastest standalone algorithm.

D.6 ZBLL would stop being 493 unrelated visual cases

The relative-permutation representation

$$\rho_4 \in A_4$$

decomposes the edge-oriented last layer into

corner state + one of 12 relative edge states.

That means a ZBLL case can be indexed structurally as

$$[\text{corner chunk}] \times [A_4 \text{ edge token}].$$

Rather than treating the algset as hundreds of arbitrary patterns, each corner family can be organized around the same twelve tetrahedral edge transformations.

This has several implications.

First, recognition becomes generative rather than purely photographic. You recognize:

1. The corner-orientation/permutation frame.
2. The relative placement of two edges.
3. The corresponding A_4 token.

Second, algorithm learning gains a common syntax. The same twelve edge relationships recur across corner subsets.

Third, mirrors, inverses, y-conjugates, and related cases become easier to organize mathematically.

Fourth, the method begins to address the issue you previously identified with ZBLL “radicals”: the building blocks exist, but their arrangement does not immediately reveal the whole algorithm. The compiler provides part of the missing grammar. It associates each structural state with a transformation operator, not merely a memorized string.

It would not make optimal ZBLL algorithms freely derivable on demand, but it could make their state structure much less opaque.

D.7 Full color neutrality would extend through the entire solve

Ordinary color neutrality is often discussed primarily as a cross-selection skill. Full compiler mastery would make the entire solution color neutral.

All information would be encoded relative to a selected frame:

$$\Phi = (D, F).$$

Absolute colors would be replaced by relative sectors, directed color differences, slot indices, and conjugation variables. The same internal transformation would apply regardless of whether the cross color was white, yellow, blue, green, red, or orange.

In mature form, the process would be:

$$\text{physical colors} \longrightarrow \text{relative frame} \longrightarrow \text{frame-invariant state code} \longrightarrow \text{motor program}.$$

This could eliminate much of the cognitive penalty normally associated with switching cross colors. A given ZBLL relational state would feel like the same state in all 24 cube orientations.

The deeper implication is that the solver would no longer reason primarily in colors. The solver would reason in **relations, orientations, and group actions**.

D.8 The cube would become external working memory

A naive approach attempts to memorize the complete initial state and propagate it mentally. The adjoint method avoids that by treating the untouched cube as a read-only database.

Instead of carrying the future LL pieces forward, you ask:

Which current stickers answer the future recognition question?

Mathematically, for an observable ϕ and prefix G ,

$$\phi(Gx_0) = (G^*\phi)(x_0),$$

where

$$G^*\phi = \phi \circ G$$

is the pullback of the observable.

You transform the **question**, not the entire state.

This has an important cognitive implication: full mastery would not require holding all 65.23 bits of the cube state consciously. It would use:

- Structured representations.
- State compression.
- Dynamic coordinate frames.
- Deferred observation.

- External visual memory.
- Parity constraints.
- Holistic macro transformations.
- Aggressive deletion of no-longer-live variables.

This is analogous to register allocation in a compiler. Information is loaded only when needed and discarded immediately after its final use.

The live symbolic load becomes approximately

$$8 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 5,$$

rather than a persistent fourteen- or sixteen-letter state.

D.9 It would demonstrate extreme expert chunking

At the beginning, each transformation would be explicit:

$$S \mapsto A, \quad B \mapsto J, \quad T \mapsto T, \quad \dots$$

Full mastery would require these individual substitutions to collapse into a single perceptual event:

$$\text{constellation} \xrightarrow{A} \text{successor constellation}.$$

Similarly, the solver would no longer transform five independent predictor letters. The solver would see a triangle-plus-bar glyph move through an inverse algorithm flow.

The relevant chunks would be:

- Four-pair F2L constellations.
- Extraction and insertion flow fields.
- Three-edge EO glyphs.
- Five-probe ZBLL predictor glyphs.
- Twelve tetrahedral A_4 tokens.
- Complete motor sequences such as

$$[C][A_1][A_2][A_3][Z][L].$$

This would be a striking example of expertise not increasing raw working-memory capacity, but changing what counts as one item in working memory.

A novice sees eight letters. An expert sees one geometric configuration.

A novice sees ten moves. An expert feels one motor gesture.

A novice sees four LL edges. An expert sees one element of A_4 .

Broad transfer to unrelated intellectual abilities should not be assumed. But strong near-domain transfer would be plausible in:

- Blindfold tracing.
- Mental cube simulation.
- Algorithm inversion.
- Commutator reasoning.
- FMC insertion tracking.
- Learning new algsets.
- Recognizing conjugacy and symmetry.
- Understanding cubic-group representations.

D.10 The solver would develop bidirectional mental simulation

Most speedcubing expertise is strongly forward-directed:

$$x \mapsto A(x).$$

The adjoint method demands equal fluency with

$$s_{\text{future}} \mapsto A^{-1}(s_{\text{future}}).$$

That means every algorithm would be known in three forms:

motor sequence, forward state flow, inverse probe flow.

Full mastery would likely change how algorithms are mentally represented. You would no longer know $RUR'U'$ merely as a fingertrick. You would know it as a complete spatial operator with an inverse flow field.

This could make learning new algorithms faster. An unfamiliar sequence could be decomposed into known transformations:

$$T_A = T_{M_k} \circ \dots \circ T_{M_2} \circ T_{M_1}.$$

Instead of memorizing every state effect independently, you would construct it from a library of trigger maps. Over time, frequently occurring compositions would themselves become chunks.

This is closer to acquiring a grammar than accumulating phrases.

D.11 The solve could become a self-indexing motor chain

Retaining forty or fifty individual moves would be impractical. Retaining six contextualized chunks is much more plausible:

$$[C][A_1][A_2][A_3][Z][L].$$

Moreover, each chunk generates the cue for the next one.

After executing C , the cube physically presents the expected starting state of A_1 . After executing A_1 , it presents the cue for A_2 , and so forth.

Thus the sequence is not stored as an unstructured list. It is a linked chain:

$$C \xrightarrow{x_1} A_1 \xrightarrow{x_2} A_2 \xrightarrow{x_3} A_3 \xrightarrow{x_4} Z \xrightarrow{x_5} L.$$

The physical cube acts as a sequence pointer.

That greatly reduces retention burden. Rather than recalling “what comes fourth,” you execute the action associated with the predicted state now in front of you.

D.12 Errors could be localized with compiler checkpoints

A mature system should not merely predict the final solution. It should emit expected intermediate invariants.

An algorithm card could contain

$$\text{Card}(A) = [\text{moves}, \text{successor cue}, \text{passive-pair map}, \text{EO map}, \text{probe map}].$$

During execution, each macro would have a lightweight checksum:

- Expected solved-slot mask.
- Expected location of the next pair.
- Expected orientation of one reference sticker.
- Expected D-phase.
- Expected AUF or frame.
- Expected ZBLS recognition landmark.

This allows error localization:

$$\text{predicted cue} \neq \text{observed cue} \implies \text{compiler state has desynchronized.}$$

The optimal response would not necessarily be to force the remaining compiled sequence. A robust solver would immediately switch into ordinary reactive solving.

Thus full mastery would include two simultaneous modes:

compiled primary mode + reactive exception handler.

This is analogous to speculative execution in computing: use the fast predicted path, but retain the ability to recover when the prediction fails.

D.13 Training would separate into distinct engineering problems

Conventional practice often consists of repeated full solves. That would be insufficient for this method.

Training would need separate modules for:

Parsing:	acquiring the relevant scramble state,
Compilation:	selecting the six macro sequence,
Simulation:	applying forward and inverse maps,
Retention:	holding the motor chain,
Execution:	performing the chain smoothly,
Verification:	checking predicted landmarks,
Recovery:	resynchronizing after an error.

A solver could therefore have distinct measured abilities:

- Cross-selection latency.
- Pair-constellation acquisition accuracy.
- Passive-pair transformation accuracy.
- EO-triad pullback time.
- Predictor-quintet pullback time.
- A_4 decoding latency.
- Full-program retention accuracy.
- Clean-program execution time.
- Recovery success rate.

This would make training much more diagnostic. A bad solve could be attributed to a specific subsystem rather than vaguely described as poor lookahead.

D.14 It could create an entirely new software ecosystem

A practical implementation would require tools capable of:

- Parsing exact algorithms.
- Generating corner and edge permutation maps.
- Computing inverse probe maps.
- Producing dynamic Speffz drills.
- Generating EO-triad exercises.
- Generating predictor-quintet exercises.
- Measuring visual probe dispersion.
- Computing passive-pair disturbance.
- Ranking alternative algorithms by cognitive and motor cost.
- Constructing personalized compiler-friendly algsets.
- Testing full-solve predictions against a cube simulator.

A solver's algset could be optimized to that individual's:

- Preferred fingertricks.
- Dominant hand.
- Rotation tolerance.
- Recognition style.
- Strongest visual fields.
- Color-neutral ability.
- Maximum reliable probe span.
- Forward-versus-inverse transformation speed.

Rather than everyone learning one supposedly universal “best” algorithm, the software could generate a personal instruction set.

The analogy is strong:

Cube system	Compiler analogue
Scramble acquisition	Parsing
Speffz/relative state	Intermediate representation
F2L/ZB algorithm library	Instruction set
Pair-order selection	Scheduling
Probe liveness	Register allocation
Algorithm selection	Code generation
Finger execution	Processor execution
Visual checkpoint	Runtime assertion
Reactive fallback	Exception handling

D.15 It could establish a new speedcubing research program

The method raises concrete empirical questions:

- How many permutation maps can humans automatize?
- Are inverse probe maps harder than forward piece maps?
- Can five probes be transformed as one spatial chunk?
- What probe geometries minimize error?
- How quickly can a person acquire a four-pair constellation?
- Does relational color coding outperform absolute color coding?
- What is the optimal balance between motor and cognitive algorithm cost?
- How low must the compilation error rate be to improve averages?

These are experimentally testable. A training platform could collect:

- Response time.
- Transformation accuracy.
- Learning curves.
- Error categories.
- Probe visibility.
- Chunking transitions.
- Retention limits.
- Performance under inspection pressure.

The result could be valuable not only as a speedsolving method, but as a controlled case study of:

- Expert memory.
- Mental rotation.
- Structured working memory.
- Procedural compilation.
- Forward versus inverse simulation.
- Perceptual chunk formation.
- Human execution of finite-state policies.

For you, this could become an original speedcubing research and software project rather than merely a personal solving technique.

D.16 The method could generalize beyond CFOP and ZB

The underlying principle is not specific to ZBLL:

Pull back only the future observables required for the next decision.

For any deterministic puzzle state x , planned prefix G , and future recognition function ϕ ,

$$\phi(Gx) = (G^* \phi)(x).$$

That applies whenever it is cheaper to transform a small recognition query backward than to propagate a large state forward.

Possible applications include:

- Predicting OLL or PLL during F2L.
- Planning Roux CMLL and LSE information.
- Tracking EO through block building.
- Predicting corner permutation in Petrus or ZZ.
- Tracking commutator targets in blindfold solving.
- Evaluating insertions in fewest moves.
- Planning centers, edge groups, or parity states on larger cubes.
- Other permutation puzzles with staged recognition.

The Adjoint ZB Compiler could therefore be the first instance of a more general **adjoint inspection paradigm**.

D.17 It would redefine what “lookahead” means

In conventional discourse, strong lookahead means seeing the next step while executing the current step.

Under full mastery, lookahead becomes much broader:

lookahead $\longrightarrow$ offline state prediction and program synthesis.

You would not merely see one pair ahead. You would know the terminal state consequences of each algorithm choice before beginning the solve.

The distinction would be:

- **Local lookahead:** identify the next object.
- **Trajectory lookahead:** predict several successive states.
- **Compiler lookahead:** select and encode the entire action sequence.
- **Adjoint lookahead:** infer future recognition outcomes from present targeted observations.

The final category is qualitatively different from what speedcubers normally call lookahead.

D.18 It could shift the speed limit from cognition to biomechanics

At present, a solve is limited by both cognitive and physical processes. If all decision-making were removed from the timed solve, the primary limits would become:

- Finger acceleration.
- Regrip latency.
- Lockup probability.
- Turning precision.
- Transition ergonomics.
- Total move count.
- Ability to retain and launch motor chunks.

A solver with somewhat lower peak TPS could outperform a faster turner by never pausing.

The relevant quantity is not maximum TPS but effective TPS:

$$\text{ETPS} = \frac{\text{total moves}}{\text{execution time} + \text{pause time}}.$$

If pause time approaches zero,

$$\text{ETPS} \rightarrow \text{physical TPS}.$$

Full compiler mastery would therefore make algorithm ergonomics and execution mechanics even more important. Once cognition is removed from the timed critical path, every lockup and regrip becomes a larger fraction of total solve time.

D.19 It would enable deliberate last-layer steering

Once the solver can predict ZBLS and ZBLL consequences, F2L algorithms can be chosen partly to control the last layer.

The planner could evaluate

$$A_i \mapsto \text{future ZBLS} \mapsto \text{future ZBLL}.$$

This permits several levels of steering:

Passive steering Choose among otherwise equivalent F2L algorithms based on the ZBLL they naturally produce.

Biased steering Accept a small F2L cost to obtain a preferred ZBLL family.

Active steering Use special multislotting or last-pair algorithms to deliberately force favorable LL structure.

Full-path steering Select the cross, pair order, F2L variants, and ZBLS as one coordinated control sequence.

This could blur the boundary between F2L and last layer. The entire solve becomes one transformation path rather than separate CFOP phases.

At the extreme, the best path might not correspond cleanly to “cross, pair, pair, pair, ZBLS, ZBLL.” Those are simply convenient macro boundaries. A compiler could eventually use:

- X-crosses.
- Double X-crosses.
- Multislotting.
- Pseudoslotting.
- Keyhole insertions.
- Pair-preserving transformations.
- Nonstandard ZBLS entries.
- Direct transitions into favorable ZBLL states.

Thus the method could evolve from a full-solve predictor into a genuinely integrated solving method.

D.20 The greatest benefit would probably be averages, not lucky singles

A very fast single can result from:

- An unusually easy cross.
- Free pairs.
- A skip.
- A favorable last layer.
- An accidental cancellation.

Full compiler mastery is most valuable when the scramble is not unusually favorable. It attempts to extract a smooth executable path from every scramble.

Therefore its most characteristic signature would likely be:

- Fewer pauses.
- Fewer disastrous solves.
- Smaller standard deviation.
- Better mean-of-100 and mean-of-1000 performance.
- Less dependence on accidental visibility.
- More reproducible turning rhythm.

In other words, it would industrialize the solve. The goal would not merely be occasional brilliance, but consistently converting arbitrary states into smooth motor programs.

D.21 Full mastery would not imply several things

It would be important not to overstate the result.

It would not automatically imply:

- Finding the shortest solution.
- Finding the fastest possible solution.
- Using the fewest moves.
- Never making execution errors.

- Being faster than every reactive solver.
- That exhaustive search can be completed during inspection.
- That all 3CE3 states are consciously memorized.
- That broad intelligence or memory capacity has increased.
- That the method is practical for every person.
- That inspection acquisition is physically easy.

The largest unresolved bottleneck may ultimately be **visual state acquisition**, not transformation.

The solver must inspect enough stickers to determine:

- Four F2L pair states.
- The selected cross.
- Three pulled-back EO values.
- Three pulled-back corner frames.
- Two pulled-back edge values.
- Relevant parity and frame information.

Even if all transformations are instantaneous, those facelets must still be visually accessed without losing previously compiled information. A standardized scan path and probe-local algorithm set may therefore matter as much as the mathematics.

The deepest implication

Full mastery would change the nature of a 3CE3 speedsolve.

It would no longer primarily be a real-time search problem. It would become a deterministic program synthesized before the timer starts:

scramble $\longrightarrow$ compressed state $\longrightarrow$ six-chunk program $\longrightarrow$ physical execution.

The most important achievement would not be memorizing more algorithms. It would be constructing a mental representation in which:

- Piece states are relational rather than absolute.
- Algorithms are operators rather than strings.
- Future recognition is converted into present observation.
- Working memory is allocated according to information liveness.
- The entire solve becomes a self-indexing motor chain.

At that point, you would not merely be “looking farther ahead” than other solvers. You would be performing a fundamentally different computational task: **compiling the solve rather than discovering it while executing it.**

E Mathematical and Algorithmic Analysis of AZBC

Yes. AZBC can be analyzed in essentially the same spirit as merge sort, quicksort, graph search, a compiler, or a real-time planning algorithm. The main difference is that **ordinary asymptotic notation alone is almost useless for a fixed 3×3 cube**.

AZBC takes an untouched cube state x_0 and compiles the six-macro program

$$M(x_0) = [C][A_1][A_2][A_3][Z][L],$$

where the prefix solves the cross and first three pairs, Z is the exact ZBLS algorithm, and L is the exact ZBL algorithm.

Its postcondition is

$$(L \circ Z \circ A_3 \circ A_2 \circ A_1 \circ C)(x_0) = x_{\text{solved}}.$$

The right way to analyze it is along several axes:

1. classical asymptotic complexity;
2. exact symbolic-operation count;
3. work-span parallel complexity;
4. visual query complexity;
5. information-theoretic efficiency;
6. online-offline time/space tradeoffs;
7. search complexity and solution optimality;
8. correctness and loop invariants;
9. error propagation and real-time reliability.

E.1 What kind of algorithm is AZBC?

AZBC is best classified as a:

deterministic, adaptive, bidirectional, table-driven compiler

operating over a finite permutation state space.

It contains both forward and backward dataflow:

F2L pair states move forward,

while

future recognition questions move backward.

The backward step is justified by the pullback identity

$$\chi_{Gx}(s) = \chi_x(G^{-1}s),$$

meaning that the sticker that will appear at future position s after prefix G can be determined by inspecting position $G^{-1}s$ on the untouched scramble.

Computationally, AZBC resembles several familiar objects at once:

Computer-science concept	AZBC analogue
Compiler front end	Frame selection and Speffz acquisition
Intermediate representation	$S_k = (\Phi, m_k, \delta_k, Q_k, P_k)$
Register allocation	Live-symbol schedule $8 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 5$
Dead-variable elimination	Delete a pair code after compiling its macro
Backward slicing	Pull only needed recognition probes backward
Partial evaluation	Specialize the future solve to the observed scramble
Memoization	Store exact algorithm transition cards

Computer-science concept	AZBC analogue
Instruction fusion	Learn superchunks such as $A_i A_j$
Finite-horizon planning	Choose C, A_1, A_2, A_3 using a weighted cost
Code generation	Output six executable motor chunks
Anytime algorithm	Fall back to the longest reliable prefix near the deadline

The state-machine formulation in the document already makes this computational interpretation explicit.

E.2 The ordinary asymptotic answer: $\Theta(1)$

For a physical 3×3 cube:

- there are always eight corners;
- there are always twelve edges;
- there are always four F2L pairs;
- the prefix always contains four macros C, A_1, A_2, A_3 ;
- the EO sensor always has three probes;
- the ZBLL predictor always has five probes.

Therefore, under the usual RAM model,

$$T_{\text{AZBC}} = \Theta(1), \quad S_{\text{AZBC}} = \Theta(1).$$

This is formally correct for exactly the same reason that any algorithm acting on a fixed 54-facelet object has constant asymptotic complexity.

It is also uninformative. It cannot distinguish:

- a 10-operation implementation from a 10-million-operation implementation;
- a 5-symbol working memory from a 16-symbol working memory;
- a serial procedure from a highly parallel visual transformation;
- a direct suffix table from an adjoint compiler.

If the input were instead supplied as a scramble string of length n , constructing x_0 from that string would require

$$\Theta(n)$$

move applications, after which the actual fixed-cube AZBC compilation would still be $\Theta(1)$.

To obtain a meaningful analysis, we need parameters internal to the architecture.

E.3 A useful parameterized model

Define:

	Symbol	Meaning	AZBC value
	r	Number of exact F2L pair codes initially tracked	4
	m	Number of prefix macros through which probes are transported	4
	p_E	Number of EO probe tokens	3
	p_L	Number of ZBLL predictor tokens	5
	b_j	Number of candidate macros examined at decision stage j	Policy-dependent

	Symbol	Meaning	AZBC value
	c_j	Cost of evaluating one candidate at stage j	Policy-dependent
	σ	Number of entries in corner-plus-edge sticker maps	$24 + 24 = 48$
	ℓ_A	Face-turn length of macro A	Algorithm-dependent

A primitive **token transport** is one application of a stored macro permutation to one Speffz token:

$$s \mapsto \tau_A(s).$$

Because τ_A is stored as an indexed table, one token transport is treated as $O(1)$. Algorithms are exact permutations of the corner- and edge-sticker alphabets, and compositions follow

$$\tau_{AB} = \tau_B \circ \tau_A.$$

E.4 Exact forward-compilation complexity

Initially there are r pair codes, with two Speffz tokens per pair.

For $r = 4$, AZBC transports:

$$8$$

tokens through the cross,

$$6$$

passive tokens through A_1 ,

$$4$$

through A_2 , and

$$2$$

through A_3 .

Therefore,

$$W_{\text{F2L}} = 8 + 6 + 4 + 2 = 20.$$

This exact count is stated in the architecture.

E.4.1 Recurrence form

For an AZBC-like compiler tracking r pairs, the forward work satisfies

$$F(r) = 2r + F(r-1), \quad F(0) = 0.$$

Solving the recurrence,

$$\begin{aligned} F(r) &= 2r + 2(r-1) + \cdots + 2 \\ &= 2 \sum_{i=1}^r i \\ &= r(r+1). \end{aligned}$$

Thus,

$$F(r) = r(r+1) = \Theta(r^2).$$

For the actual cube,

$$F(4) = 4(5) = 20.$$

This is the closest direct analogue to deriving the $T(n)$ recurrence for merge sort, except AZBC's forward register contracts by two tokens at each stage rather than dividing an array into halves.

E.5 Complexity of the adjoint probes

Suppose a glyph containing p tokens is pulled backward through m exact macros.

At the token level,

$$P(m, p) = p + P(m - 1, p), \quad P(0, p) = 0,$$

so

$$\boxed{P(m, p) = mp.}$$

E.5.1 EO triad

The three-token EO glyph passes backward through

$$A_3^{-1}, A_2^{-1}, A_1^{-1}, C^{-1}.$$

Therefore,

$$W_{\text{EO}} = p_E m = 3 \cdot 4 = 12.$$

E.5.2 ZBLL quintet

The predictor contains five tokens. The Z^{-1} portion is already compiled into the ZBLS card as

$$Q_Z = Z^{-1} Q_{\text{final}},$$

so online it only needs to pass through the four-prefix inverse:

$$W_{\text{ZBLL}} = p_L m = 5 \cdot 4 = 20.$$

The stored quintet and its exact-sequence dependence are central parts of the architecture.

E.5.3 Total token-transport work

Consequently,

$$\begin{aligned} W_{\text{maps}} &= W_{\text{F2L}} + W_{\text{EO}} + W_{\text{ZBLL}} \\ &= 20 + 12 + 20 \\ &= \boxed{52}. \end{aligned}$$

In parameterized form,

$$\boxed{W_{\text{maps}}(r, m, p_E, p_L) = r(r + 1) + m(p_E + p_L).}$$

For an architecture in which $m = r$,

$$W_{\text{maps}} = r(r + 1) + r(p_E + p_L).$$

If probe sizes also scale like $O(r)$, the generalized compiler remains

$$\Theta(r^2)$$

in token-transport work.

The document independently arrives at the 52-lookup serial budget and argues that this is too large for a conscious 15-second process.

E.6 Work-span analysis: the key parallelism result

The 52 count is the **work**

$$W,$$

but it need not equal the serial cognitive depth if several tokens are transformed together as one spatial object. Under the intended holistic model:

- the complete live F2L constellation is transformed once through each of four forward macros;
- the EO triangle is transformed once through each of four inverse macros;
- the predictor triangle-plus-bar is transformed once through each of four inverse macros.

Therefore, the transition **span** is

$$S = 4 + 4 + 4 = 12.$$

So the available parallelism is

$$\frac{W}{S} = \frac{52}{12} = \frac{13}{3} \approx 4.33.$$

Thus the central work-span profile is

$$W = 52, \quad S = 12, \quad W/S \approx 4.33.$$

This gives a precise interpretation of the architecture's claim that a mature solver must transform a constellation or glyph as a single scene rather than perform independent letter lookups.

It is analogous to SIMD or vector processing:

$$\begin{array}{ccc} \text{serial implementation:} & \longrightarrow & \text{holistic implementation:} \\ \text{transform each token separately} & & \text{transform the whole spatial register} \end{array}$$

The 12-scene count is still not the entire critical path. Recognition, physical viewing, algorithm retrieval, and pair selection occur between those transformations. It is specifically the **state-transition span**.

E.7 Total online time complexity

The 52 transports do not include candidate search.

Let b_j be the number of candidate exact macros tested at stage j , and let c_j be the cost of:

- normalization;
- passive-pair prediction;
- motor-cost evaluation;
- EO disturbance evaluation;
- probe-locality evaluation;
- risk scoring.

Then a useful general expression is

$$T_{\text{online}} = \Theta \left(r(r+1) + m(p_E + p_L) + \sum_j b_j c_j + T_{\text{frame}} + T_{\text{observe}} + T_{\text{lookup}} \right).$$

The document's cost function explicitly includes motor, passive-pair, EO, probe, and risk terms, and notes that the move-optimal algorithm need not be inspection-optimal.

E.7.1 Fixed-policy implementation

Suppose the system stores exactly one deterministic action for every recognized state. Then each macro decision is one indexed lookup:

$$b_j = 1, \quad c_j = \Theta(1).$$

The computational portion becomes

$$\Theta(r^2 + m(p_E + p_L)).$$

For the fixed 3×3 ,

$$\Theta(1).$$

E.7.2 Greedy adaptive implementation

The current pseudocode chooses

$$A_j = \arg \min_A J(A, Q, \text{prefix})$$

at each stage.

If it examines b_j candidates independently,

$$T_{\text{greedy}} = \Theta\left(\sum_j b_j c_j\right).$$

E.7.3 Exhaustive global planning

If candidate choices have branching factors $b_1, b_2, \dots, b_k$, exhaustive search explores

$$1 + b_1 + b_1 b_2 + \dots + \prod_{i=1}^k b_i$$

nodes.

For uniform branching factor b ,

$$T_{\text{exhaustive}} = \Theta(b^k).$$

Here $k = 3$ after cross selection, so the physical problem remains bounded, but the mental branching burden can still be prohibitive.

E.7.4 Beam search

With beam width w , average branching factor b , and depth k ,

$$T_{\text{beam}} = \Theta(kwb c),$$

with search memory

$$S_{\text{beam}} = \Theta(w).$$

This would be a natural software implementation for adaptive pair-order planning.

E.8 AZBC as a shortest-path or dynamic-programming problem

The prefix-selection problem can be formalized as deterministic finite-horizon optimal control.

Let

$$S_j = (\Phi, m_j, \delta_j, Q_j, P_j)$$

be the state after j pair macros, and let $\mathcal{A}(S_j)$ be the legal exact actions. Define a stage cost

$$c(S_j, A)$$

containing motor, pair-distortion, probe, EO, and risk costs.

Then the globally optimal value function satisfies the Bellman recurrence

$$V_j(S_j) = \min_{A \in \mathcal{A}(S_j)} [c(S_j, A) + V_{j+1}(T_A(S_j))].$$

The terminal cost should include the difficulty of:

- the resulting $q_\star$;
- the EO pullback;
- the selected ZBLS predictor;
- the final ZBLL recognition;
- the completed move sequence.

This exposes an important distinction:

$$\text{local } \arg \min J(A) \neq \text{global optimum}$$

unless the local score already contains the exact cost-to-go V_{j+1} .

Therefore:

$$\boxed{\text{AZBC correctness does not imply AZBC cost optimality.}}$$

The present architecture provides an exact method for producing a valid solve, but it does not prove:

- minimum move count;
- minimum inspection time;
- minimum visual-query cost;
- a bounded approximation ratio relative to the globally best six-macro plan.

A useful empirical optimality statistic would be the per-scramble regret

$$R(x) = J(M_{\text{AZBC}}(x)) - J(M^\star(x)),$$

or the approximation ratio

$$\alpha(x) = \frac{J(M_{\text{AZBC}}(x))}{J(M^\star(x))}.$$

An offline dynamic program could calculate $M^\star$ on a restricted fixed-frame state graph and then evaluate the human policy against it.

E.9 Correctness theorem

E.9.1 Assumptions

Let the following conditions hold:

1. Initial Speffz acquisition is exact.
2. Every stored macro card contains the correct sticker permutations.
3. Every chosen F2L macro satisfies its stated precondition and solves its selected pair.
4. Solved-slot, AUF, frame, and pseudo- D phases are represented consistently.

5. The ZBLS lookup is complete and returns the exact stored sequence.
6. The ZBLL lookup is complete and returns the exact algorithm including AUFs.
7. All physical probe observations are read correctly.
8. The exact algorithm variant chosen during inspection is the one later executed.

The document identifies essentially these same invariants.

E.9.2 Prefix invariant

Let

$$G_j = A_j \circ A_{j-1} \circ \dots \circ A_1 \circ C$$

and let Π_j project a physical cube state onto the exact codes of the remaining unsolved F2L pairs. The loop invariant is

$$\boxed{Q_j = \Pi_j(G_j x_0)}.$$

It also requires that $G_j x_0$ has:

- a solved cross;
- exactly j compiled F2L pairs solved;
- a solved-slot mask and phase state equal to the internal record.

Base case After choosing C ,

$$Q_0 = T_C(Q_{\text{initial}}).$$

Because T_C is the exact sticker permutation of C ,

$$Q_0 = \Pi_0(Cx_0).$$

Inductive step Assume

$$Q_j = \Pi_j(G_j x_0).$$

The normalizer and lookup choose an exact macro A_{j+1} matching the active pair's state. Applying its map to every passive pair gives their exact states in

$$A_{j+1}G_j x_0.$$

Deleting the newly solved pair therefore leaves

$$Q_{j+1} = \Pi_{j+1}(A_{j+1}G_j x_0).$$

So the invariant holds by induction.

After three pair decisions,

$$q_\star$$

is exactly the remaining pair state at the ZBLS boundary.

E.9.3 EO correctness

Let $E(q_\star)$ be the future three-position EO sensor.

AZBC computes

$$E_0 = G_3^{-1}E(q_\star).$$

By the pullback identity,

$$\text{Obs}_{E(q_\star)}(G_3 x_0) = \text{Obs}_{E_0}(x_0).$$

Thus the three observed bits are exactly the three future EO bits.

The fifth unsolved-edge orientation is already known from q_* , and the remaining unknown orientation is reconstructed from global edge parity. Three probes are therefore sufficient.

The resulting ZBLS lookup selects an exact Z that solves the final pair and orients all LL edges.

E.9.4 ZBLL correctness

Let Q_{final} be the canonical three-corner-plus-two-edge sensor in the post-ZBLS state.

Each ZBLS card stores

$$Q_Z = Z^{-1}Q_{\text{final}}.$$

AZBC then computes

$$Q_0 = G_3^{-1}Q_Z = G_3^{-1}Z^{-1}Q_{\text{final}}.$$

Since

$$G_4 = Z \circ G_3,$$

we have

$$Q_0 = G_4^{-1}Q_{\text{final}}.$$

Therefore,

$$\text{Obs}_{Q_{\text{final}}}(G_4x_0) = \text{Obs}_{Q_0}(x_0).$$

The three corner frames determine the entire corner permutation and orientation, and two edge observations determine the edge permutation by parity.

The selected L therefore solves the exact state G_4x_0 , proving

$$(L \circ G_4)x_0 = x_{\text{solved}}.$$

Hence:

Under the assumptions above, AZBC is correct for every covered legal state.

E.9.5 Termination

The online loop has exactly three iterations. Every candidate set and lookup table is finite. Therefore AZBC terminates whenever every reachable state has at least one legal macro card.

If the card library or normalizer does not cover every reachable case, AZBC is a **partial algorithm** rather than a total algorithm. The mathematical architecture specifies how complete coverage should work; universal human implementation has not yet been empirically demonstrated.

E.10 Online space complexity

The live state-token schedule is

$$8 \longrightarrow 6 \longrightarrow 4 \longrightarrow 2 \longrightarrow 3 \longrightarrow 5.$$

Thus the maximum live symbolic state register is

8 tokens.

The six compiled motor chunks are stored separately as the output program

$$[C][A_1][A_2][A_3][Z][L].$$

In parameterized form,

$$S_{\text{state}} = \Theta(\max\{2r, p_E, p_L\}).$$

Including the growing macro stack,

$$S_{\text{online}} = \Theta(\max\{2r, p_E, p_L\} + m).$$

For the actual 3×3 , this is still $\Theta(1)$.

The destructive deletion of pair codes is algorithmically important. AZBC does not retain every intermediate piece state. It performs

exact state $\longrightarrow$ committed macro

and reclaims the state register. This is analogous to liveness-based register reuse in a compiler.

E.11 Logical visual-query complexity

Treat the untouched scramble as an immutable oracle:

$$\text{Observe}_{x_0}(s) = \chi_{x_0}(s).$$

AZBC issues adaptive queries against this oracle.

Ignoring the frame and cross scan, the raw logical observations are:

$$\begin{aligned} Q_{\text{pair}} &= 8, \\ Q_{\text{EO}} &= 3, \\ Q_{\text{ZBLL}} &= 5. \end{aligned}$$

Thus

$$Q_{\text{logical}} = 8 + 3 + 5 = 16.$$

This does **not** mean only 16 physical glances are required. Locating a cubie or reading an entire oriented corner frame may require viewing several facelets. It is a symbolic query count.

E.11.1 Adaptive query rounds

The core compiler has three main observation rounds:

1. acquire the four-pair constellation;
2. after q_* and the prefix are known, query EO;
3. after Z is known, query the ZBLL predictor.

The third query set cannot generally be formed before the second round because

$$Q_Z = Z^{-1}Q_{\text{final}}$$

depends on which exact ZBLS algorithm was selected.

This makes AZBC an **adaptive decision-tree algorithm** rather than a one-pass scanner.

E.12 A new bound on EO–quintet probe overlap

The architecture defines

$$N_{\text{probe}}(Z) = |E(q_*) \cup Z^{-1}Q_{\text{final}}|.$$

The EO sensor contains three edge positions.

The predictor contains:

- three corner positions;

- two edge positions.

Cube permutations preserve piece type, so the EO edge probes can overlap only with the two edge components of the quintet. Therefore,

$$0 \leq |E(q_\star) \cap Z^{-1}Q_{\text{final}}| \leq 2.$$

Consequently,

$$\begin{aligned} N_{\text{probe}}(Z) &= 3 + 5 - |E(q_\star) \cap Z^{-1}Q_{\text{final}}| \\ &\in \{6, 7, 8\}. \end{aligned}$$

Hence:

$$\boxed{6 \leq N_{\text{probe}}(Z) \leq 8.}$$

This produces a strict limit on probe-overlap optimization:

$$\boxed{\text{A standard ZBLS card can save at most two of the eight late probe locations.}}$$

That is a maximum reduction of

$$\frac{2}{8} = 25\%.$$

This is a useful hard bound for the multiobjective algorithm search proposed in the document. The source establishes that overlap can be optimized offline because inverse permutations preserve intersections; the 6-to-8 bound follows additionally from corner/edge type preservation.

E.13 Information-theoretic analysis

E.13.1 Three-pair representation

For three identified pairs, the exact state count is

$$N_3 = (8)_3 3^3 (12)_3 2^3 = 95,800,320,$$

where $(n)_k$ is the falling factorial.
Therefore,

$$\log_2 N_3 \approx 26.51 \text{ bits.}$$

Six Speffz symbols have capacity

$$6 \log_2 24 \approx 27.51 \text{ bits.}$$

The redundancy is only

$$27.51 - 26.51 \approx 1.00 \text{ bit.}$$

So the six-letter representation is about

$$\frac{26.51}{27.51} \approx 96.4\%$$

information-efficient.

E.13.2 Four-pair ring: an extension of the document's calculation

Once the frame and logical pair identities are fixed, the four F2L corners can occupy ordered distinct positions in

$$(8)_4$$

ways, with 3^4 orientations. The four F2L edges have

$$(12)_4$$

ordered positions and 2^4 orientations.

Thus

$$\begin{aligned} N_4 &= (8)_4 3^4 (12)_4 2^4 \\ &= 25,866,086,400. \end{aligned}$$

Its entropy is

$$\log_2 N_4 \approx 34.59 \text{ bits.}$$

Eight Speffz letters provide

$$8 \log_2 24 \approx 36.68 \text{ bits.}$$

Therefore the initial four-pair ring has approximately

$$36.68 - 34.59 = 2.09$$

bits of representational redundancy and is roughly

$$94.3\%$$

information-efficient under this partial-state model.

So the eight-token register is not just convenient; it is reasonably close to an exact compact encoding.

E.13.3 EO triad optimality

At the ZBLS boundary, five edge-orientation bits remain:

$$(f_*, f_0, f_1, f_2, f_3),$$

subject to

$$f_* \oplus f_0 \oplus f_1 \oplus f_2 \oplus f_3 = 0.$$

The exact remaining-pair code already determines f_* . Hence the four remaining bits have one parity constraint and therefore only

$$4 - 1 = 3$$

degrees of freedom.

The number of possible EO contexts is

$$2^3 = 8.$$

Any binary-probe method therefore requires at least

$$\log_2 8 = 3$$

bits.

AZBC uses exactly three one-bit orientation probes:

$$\boxed{Q_{\text{EO}} = 3 = \text{information-theoretic minimum.}}$$

This is not merely a sufficient sensor. It is optimal in the binary edge-orientation probe model.

E.13.4 Final quintet optimality

After ZBLS, the exact edge-oriented last layer has

$$4! \cdot 3^3 \cdot |A_4| = 24 \cdot 27 \cdot 12 = 7,776$$

states.

Its entropy is

$$\log_2 7776 \approx 12.92 \text{ bits.}$$

Corner portion For three selected corner positions:

- the first observation has $4 \cdot 3 = 12$ possibilities;
- the second has $3 \cdot 3 = 9$;
- the third has $2 \cdot 3 = 6$.

Therefore,

$$12 \cdot 9 \cdot 6 = 648 = 4! 3^3.$$

This is exactly the number of complete corner permutation-orientation states.

Edge portion After the corner frame is known, the first observed edge has four possible identities and the second has three:

$$4 \cdot 3 = 12.$$

This equals

$$|A_4| = 12.$$

The other two edge placements are completed uniquely by parity.
Therefore the total number of quintet observation outcomes is

$$648 \cdot 12 = 7,776,$$

exactly equal to the full post-ZBLS state count.

Hence:

The predictor quintet is entropy-tight in its one-cubie observation model.

It contains neither missing states nor combinatorial redundancy.

Moreover, no set of four probes composed only of oriented-corner and oriented-edge observations can be sufficient.
The most informative four-probe allocation is three corners plus one edge:

$$(12 \cdot 9 \cdot 6) \cdot 4 = 2,592 < 7,776.$$

Thus at least five such probes are required, and AZBC attains that minimum:

$$Q_{\text{ZBLL}} = 5 = \text{minimum within the stated probe model.}$$

E.14 The A_4 token as an optimal edge code

Once the corner frame is known, the relative edge permutation satisfies

$$\rho = C^{-1} \circ E \in A_4.$$

Since

$$|A_4| = 12,$$

the final edge relation contains

$$\log_2 12 \approx 3.585$$

bits.

The code

$$R_2 = \rho(0)\rho(1)$$

uses two distinct symbols from a four-symbol alphabet. There are exactly

$$4 \cdot 3 = 12$$

such ordered pairs, so it is a bijection onto A_4 .

Thus the two-symbol edge token is also combinatorially exact:

$$\{\text{distinct ordered pairs from four symbols}\} \cong A_4.$$

It is not merely a mnemonic abbreviation; it is a minimal image representation of the even relative edge permutation.

E.15 Offline preprocessing complexity

AZBC moves substantial work offline.

For each exact macro A , its card stores items such as:

- 24-entry corner-sticker permutation;
- 24-entry edge-sticker permutation;
- position and orientation maps;
- inverse probe flow;
- phase changes;
- solved-slot effects;
- cognitive costs;
- predictor quintet for ZBLS.

The proposed machine record is detailed in the document.

Let:

- N be the total number of exact macro cards;
- N_Z be the number of exact ZBLS cards;
- ℓ_i be the face-turn length of card i ;
- $\sigma = 48$ be the total number of corner- and edge-sticker positions.

Generating every map from face-turn generators requires approximately

$$T_{\text{cards}} = \Theta\left(\sigma \sum_{i=1}^N \ell_i\right).$$

Because $\sigma = 48$ is fixed for the 3×3 ,

$$T_{\text{cards}} = \Theta\left(\sum_i \ell_i\right).$$

Storage is

$$S_{\text{cards}} = \Theta(\sigma N + p_L N_Z),$$

plus constant-sized metadata per card.

E.15.1 Why precomputation matters

Without a stored macro map, transporting one token through algorithm A would require applying all ℓ_A face-turn generators:

$$O(\ell_A).$$

With a stored card, it becomes one table lookup:

$$O(1).$$

Thus algorithm cards replace move-level online complexity

$$\sum_A n_A \ell_A$$

with macro-level complexity

$$\sum_A n_A,$$

where n_A is the number of live tokens crossing macro A .

That is a genuine preprocessing speedup, analogous to compiling source code or memoizing a repeatedly evaluated function.

E.16 Direct-table baseline versus AZBC

After cross plus three pairs, the exact residual suffix state has

$$9,331,200$$

possibilities. A monolithic algorithm could theoretically store

$$\text{exact residual state} \mapsto (Z, L)$$

for every one of these states.

That baseline has:

$$T_{\text{lookup}} = \Theta(1)$$

after the exact key is known, but

$$S_{\text{table}} = \Theta(9,331,200)$$

entries.

For ordinary software, nine million entries are not impossible. Therefore AZBC's central advantage is not that it asymptotically defeats a computer lookup table.

Its advantage is that it replaces one enormous, perceptually inaccessible key with:

1. an exact final-pair state;
2. an information-minimal three-bit EO gate;
3. a selected ZBLS card;
4. an information-tight five-probe final-state gate;
5. a compact A_4 token.

So AZBC is principally a solution to **human key acquisition and working-memory scheduling**, not merely computer memory consumption.

E.17 Time-space tradeoffs from superchunks

Suppose the basic library contains N maps.

Storing all possible two-macro compositions would require approximately

$$O(N^2)$$

superchunks, which quickly becomes unmanageable.

If only F frequently occurring compositions are learned, additional storage is

$$O(\sigma F),$$

while each successful use removes one online scene-transition step.

This creates a classic preprocessing tradeoff:

$$\boxed{\text{more stored compositions} \iff \text{lower online span.}}$$

The source therefore recommends frequency-based superchunks rather than exhaustive pairwise storage.

The same tradeoff appears with sensor bases:

- one basis per ZBLS card minimizes storage and selection cost;
- several bases improve visibility;
- searching all 24 bases adds online decision work.

E.18 Best-, average-, and worst-case behavior

For merge sort, the number of comparisons varies little; for quicksort, pivot quality changes the runtime dramatically.

For AZBC, the **core token-transport work is input-independent** under a fixed policy:

$$W_{\min} = W_{\text{avg}} = W_{\max} = 52.$$

What varies by scramble is:

- frame-selection time;
- cross-search effort;
- pair normalization difficulty;
- number of candidates considered;
- probe visibility and dispersion;
- algorithm retrieval latency;
- move count and ergonomics of the output;
- human error probability.

A proper average-case analysis needs a scramble distribution μ :

$$\mathbb{E}_{\mu}[T] = \sum_{x \in \mathcal{X}} \mu(x) T(x).$$

A meaningful experimental report should give:

$$\text{median}(T), \quad P_{90}(T), \quad P_{95}(T), \quad P_{99}(T), \quad \Pr(T < D),$$

rather than only the mean.

The document already recommends separate latency and accuracy distributions for each compiler stage because total inspection time alone is diagnostically weak.

No numerical average-case or worst-case human runtime can yet be inferred from the framework. Its sub-15-second schedule is explicitly an aspirational design target rather than a demonstrated universal result.

E.19 AZBC as a hard-real-time and anytime algorithm

The compiler operates under a strict deadline. Consequently, average runtime alone is insufficient.

For a deadline D , the relevant success quantity is

$$\Pr(T_{\text{compile}} \leq D \wedge M(x_0) \text{ is correct}).$$

A conventional deterministic real-time guarantee would require

$$T_{\text{WCET}} \leq D,$$

where WCET is worst-case execution time. Human cognition does not currently provide a stable enough WCET for such a guarantee.

The fallback ladder instead makes AZBC an **anytime planning algorithm**:

$$[C] \subset [C, A_1] \subset [C, A_1, A_2] \subset [C, A_1, A_2, A_3] \subset [C, A_1, A_2, A_3, Z] \subset [C, A_1, A_2, A_3, Z, L].$$

As time advances, the compiler attempts to extend the verified planning horizon. Near the deadline, it returns the longest reliable prefix rather than a speculative full solution.

This graceful degradation is algorithmically valuable: a partial but correct program is preferable to a complete but incorrectly compiled program.

E.20 Error-propagation complexity

Mathematical AZBC is deterministic and exact. Human AZBC is a noisy implementation of that deterministic computation.

Let the relevant stage-correctness events be

$$E_1, E_2, \dots, E_k.$$

Without assuming independence,

$$\Pr(\text{success}) = \prod_{i=1}^k \Pr(E_i \mid E_1, \dots, E_{i-1}).$$

This is important because errors cascade:

- a wrong pair acquisition corrupts the forward prefix;
- a wrong exact macro variant changes passive states and inverse maps;
- a wrong q_* changes the EO sensor;
- a wrong EO bit selects the wrong ZBLS;
- a wrong ZBLS changes the predictor quintet;
- a wrong corner frame or edge observation changes the ZBLL;
- an execution mistake invalidates the predicted suffix.

The deterministic algorithm-set policy is therefore not merely stylistic. Improvising between nominally equivalent algorithms invalidates cached successor and predictor maps.

E.20.1 Illustrative independent-error model

Suppose each of the 52 serial token transports is independently correct with probability p . Then

$$P_{\text{transport}} = p^{52}.$$

To obtain at least 95% success from transport operations alone,

$$p^{52} \geq 0.95,$$

so

$$p \geq 0.95^{1/52} \approx 0.999014.$$

Thus each primitive lookup would need approximately

$$\boxed{99.9014\%}$$

accuracy.

Under the 12-scene holistic model, if each scene transformation has independent accuracy q ,

$$q^{12} \geq 0.95$$

requires

$$q \geq 0.95^{1/12} \approx 0.995735,$$

or approximately

$$\boxed{99.5735\%}$$

accuracy per scene.

This model is deliberately simplified—scene errors will be correlated and scenes are more complex than single-token lookups—but it demonstrates why reducing serial failure opportunities matters for both speed and reliability.

E.20.2 Redundant EO checksum

Reading all four non-active EO positions gives one redundant parity check. If $f_\star$ is trusted, the condition

$$f_\star \oplus f_0 \oplus f_1 \oplus f_2 \oplus f_3 = 0$$

detects any odd number of EO-reading errors, including every single-bit observation error. It cannot detect an even number of flipped readings.

Thus the fourth EO probe behaves exactly like a one-bit parity checksum:

$$\text{one extra query} \iff \text{single-error detection.}$$

E.21 Formal algorithm profile

Property	AZBC
Input	Untouched legal 3×3 state x_0
Output	Six-macro executable program
Mathematical type	Deterministic adaptive transducer/compiler
State action	Exact permutations
Dataflow	Forward state propagation plus backward probe pullback
Classical runtime	$\Theta(1)$
Parameterized map work	$\Theta(r^2 + m(p_E + p_L))$
Exact token work	52
Ideal transition span	12 scene transformations
Available transition parallelism	$52/12 \approx 4.33$
Live state-register peak	8 tokens
Raw logical observations	16, excluding cross/frame scan
Core adaptive observation rounds	3
EO probe count	3, information-theoretically minimal
ZBL probe count	5, minimal in the one-cubie probe model
Late unique probe bound	6 to 8
Offline card generation	$\Theta(\sigma \sum_i \ell_i)$
Offline card storage	$\Theta(\sigma N + p_L N_Z)$
Correctness	Exact under acquisition/card/lookup assumptions
Cost optimality	Not presently guaranteed

Property	AZBC
Human runtime guarantee	Not yet established
Fault tolerance	Low without checks or fallback
Deadline behavior	Anytime degradation to a reliable prefix

E.22 The central algorithmic conclusion

The most meaningful analogue to saying

merge sort has $\Theta(n \log n)$ time and $\Theta(n)$ space

is the following statement:

**AZBC is a fixed-domain deterministic adaptive compiler
with 52 serial token transports,
12 idealized scene-transition depth,
8-token peak live state,
3 adaptive query rounds,
3 information-minimal EO probes, and
5 information-tight ZBLL probes.**

Under a generalized register model, its core transition work is

$$\Theta(r^2 + m(p_E + p_L)),$$

while candidate planning ranges from

$$\Theta\left(\sum_j b_j c_j\right)$$

for greedy search to

$$\Theta(b^k)$$

for exhaustive planning.

Its correctness follows from an exact forward-state invariant and two applications of the adjoint pullback identity. Its central unresolved issue is not mathematical correctness or information capacity, but whether the 52 primitive operations can be compressed into roughly 12 reliable holistic visual transformations under a hard real-time deadline. The document reaches essentially the same boundary: the cube mathematics is exact, while universal human attainability remains an empirical research question.

F A fully worked AZBC example solve

This example traces an entire solve through the **Adjoint ZB Compiler**:

cross $\rightarrow$ three forward-compiled F2L pairs $\rightarrow$ backward EO query $\rightarrow$ ZBLS $\rightarrow$ backward ZBLL query $\rightarrow$ ZBLL.

The scramble is deliberately constructed so that every transformation is transparent and independently verifiable. It is a legal cube state, but it is not claimed to be a randomly generated WCA scramble or a globally optimal solution.

The complete solution has 30 HTM:

$$6 + 4 + 4 + 4 + 5 + 7 = 30.$$

An exact 54-sticker permutation simulation verifies that the final sequence solves the cube.

F.1 Conventions

Use the fixed color frame

$$U = \text{white}, \quad D = \text{yellow}, \quad F = \text{green},$$

$$R = \text{red}, \quad B = \text{blue}, \quad L = \text{orange}.$$

We therefore solve a **yellow cross**, with green initially designated as front:

$$\Phi = (D = \text{yellow}, F = \text{green}).$$

All Speffz letters are interpreted relative to this frame.

The four logical F2L slots are ordered as

$$(FR, RB, BL, LF).$$

For each pair, we track:

Pair	Tracked corner sticker	Tracked edge sticker
FR	D -sticker of DFR	F -sticker of FR
RB	D -sticker of DRB	R -sticker of RB
BL	D -sticker of DBL	B -sticker of BL
LF	D -sticker of DLF	L -sticker of LF

Thus a two-letter code

$$q_i = \gamma_i \epsilon_i$$

specifies the exact oriented location of the tracked corner sticker and edge sticker.

For example,

$$JB$$

for the FR pair means:

- The D -sticker of the DFR corner is at corner Speffz position J .
- The F -sticker of the FR edge is at edge Speffz position B .

The corner and edge Speffz alphabets are separate. Thus a code such as II is unambiguous: the first I is a corner position and the second I is an edge position.

No persistent pseudo- D phase is used in this example:

$$\delta = 0$$

at every macro boundary.

F.2 Scramble

Apply the following scramble to a solved cube:

R U2 R' U' R U' R' U' F R' F' R
 L' U' L U R' U' R U L U L' U'
 D L2 B' R D' F'

As one line:

R U2 R' U' R U' R' U' F R' F' R L' U' L U R' U' R U L U L' U' D L2 B' R D' F'

The final compiled program will be

$$\mathcal{P} = [C][A_1][A_2][A_3][Z][L],$$

with

$$\begin{aligned} C &= F D R' B L^2 D', \\ A_1 &= U L U' L', \\ A_2 &= U' R' U R, \\ A_3 &= U' L' U L, \\ Z &= R' F R F' U, \\ L &= R U R' U R U^2 R'. \end{aligned}$$

The scramble was generated as the exact inverse of this program:

$$S = \mathcal{P}^{-1}.$$

Therefore,

$$\mathcal{P}(S(e)) = e,$$

where e is the solved state.

F.3 Initial cross and F2L acquisition

F.3.1 Cross-edge word

Track the yellow stickers of the four cross edges in the logical order

$$(DF, DR, DB, DL).$$

In the scrambled state, their Speffz positions are

$$X_{\text{cross}}^{(0)} = [I, H, D, X].$$

The selected cross is

$$C = F D R' B L^2 D'.$$

Under its edge-sticker permutation,

$$[I, H, D, X] \xrightarrow{C} [U, V, W, X].$$

The letters U, V, W, X are exactly the four yellow-face Speffz positions for

$$DF, DR, DB, DL.$$

Therefore the cross is solved.

One cross edge, DL , was already solved:

$$X \xrightarrow{C} X.$$

The other three are transported into their solved positions.

F.3.2 Initial four-pair ring

Before executing the cross, the exact F2L pair ring is

$$Q_{\text{initial}} = (BK, PN, HW, RL)$$

in the fixed slot order

$$(FR, RB, BL, LF).$$

Expanded:

Logical pair	Initial code
FR	BK
RB	PN
BL	HW
LF	RL

For example, the FR code BK says:

- The yellow sticker of the DFR corner is at corner position B , the U -face sticker of URB .
- The green sticker of the FR edge is at edge position K , the F -face sticker of DF .

The solver does not need to retain this verbal description. The two-letter code is the exact intermediate representation.

F.4 Transporting all four pairs through the cross

Applying the cross permutation to every pair gives

$$Q_0 = (JR, QM, RD, II).$$

The exact transformations are

$$\begin{aligned} FR : BK &\xrightarrow{C} JR, \\ RB : PN &\xrightarrow{C} QM, \\ BL : HW &\xrightarrow{C} RD, \\ LF : RL &\xrightarrow{C} II. \end{aligned}$$

This is the first major AZBC operation:

One cross macro transforms the entire four-pair constellation.

The solver now evaluates pair order in the post-cross state, not in the original scramble. The resulting state is favorable:

- BL is a direct insertion.
- RB remains simple after BL .
- LF becomes a direct insertion after RB .
- FR becomes the final ZBLS pair.

The selected pair order is therefore

$$BL \rightarrow RB \rightarrow LF \rightarrow FR.$$

F.5 Forward compilation of the first three pairs

F.5.1 First pair: BL

The active pair is

$$q_{BL} = RD.$$

Choose

$$\boxed{A_1 = U L U' L'}.$$

The active stickers transform as

$$R \xrightarrow{A_1} X, \quad D \xrightarrow{A_1} R.$$

Therefore

$$RD \xrightarrow{A_1} XR.$$

The solved code for the BL pair is XR , so the pair is solved.

At the same time, the passive pairs transform as

$$\begin{aligned} FR : \quad JR &\xrightarrow{A_1} FC, \\ RB : \quad QM &\xrightarrow{A_1} QM, \\ LF : \quad II &\xrightarrow{A_1} JE. \end{aligned}$$

Thus the full ring changes according to

$$(JR, QM, RD, II) \xrightarrow{A_1} (FC, QM, XR, JE).$$

Once A_1 is selected, the active code RD can be discarded. The remaining live register is only

$$\boxed{Q_1^{\text{live}} = (FC, QM, JE)}.$$

The symbolic load has contracted from eight letters to six.

The solved-slot mask, in the order (FR, RB, BL, LF) , is

$$m_1 = (0, 0, 1, 0).$$

F.5.2 Second pair: RB

The next active pair is still

$$q_{RB} = QM.$$

Choose

$$\boxed{A_2 = U' R' U R}.$$

Its active-sticker action is

$$Q \xrightarrow{A_2} W, \quad M \xrightarrow{A_2} N,$$

and therefore

$$QM \xrightarrow{A_2} WN.$$

The solved code for the RB pair is WN , so the pair is solved.

The passive unsolved pairs transform as

$$FR : \quad FC \xrightarrow{A_2} JB,$$

$$LF : \quad JE \xrightarrow{A_2} IE.$$

The already solved BL pair remains solved.

Thus,

$$(FC, QM, XR, JE) \xrightarrow{A_2} (JB, WN, XR, IE).$$

After compiling A_2 , discard the RB code. The live register contracts to

$$\boxed{Q_2^{\text{live}} = (JB, IE)}.$$

The solved-slot mask is now

$$m_2 = (0, 1, 1, 0).$$

F.5.3 Third pair: LF

The active pair is

$$q_{LF} = IE.$$

Choose

$$\boxed{A_3 = U' L' U L}.$$

The active stickers transform as

$$I \xrightarrow{A_3} U, \quad E \xrightarrow{A_3} F.$$

Therefore,

$$IE \xrightarrow{A_3} UF.$$

The solved LF code is UF .

The remaining FR pair is fixed by this macro:

$$JB \xrightarrow{A_3} JB.$$

Thus,

$$(JB, WN, XR, IE) \xrightarrow{A_3} (JB, WN, XR, UF).$$

After compiling A_3 , only one pair remains live:

$$\boxed{Q_3^{\text{live}} = (JB)}.$$

The solved-slot mask is

$$m_3 = (0, 1, 1, 1).$$

F.5.4 Complete forward-state table

State	Applied macro	FR	RB	BL	LF	Solved slots
x_0	—	BK	PN	HW	RL	none
x_C	C	JR	QM	RD	II	cross
x_1	A_1	FC	QM	XR	JE	BL
x_2	A_2	JB	WN	XR	IE	RB, BL
x_3	A_3	JB	WN	XR	UF	RB, BL, LF

The forward F2L payload has contracted as

$$8 \rightarrow 6 \rightarrow 4 \rightarrow 2$$

Speffz letters.

This completes the **forward compilation phase**.

F.6 Decoding the remaining pair

At x_3 , the remaining FR pair has code

$$\boxed{JB}.$$

The corner letter J means that the yellow sticker of the DFR corner is at the F -face sticker of UFR .

The edge letter B means that the green sticker of the FR edge is at the U -face sticker of UR .

Concretely, immediately before ZBLS:

F.6.1 UFR corner

Facelet	Color
U	green
R	red
F	yellow

This is the DFR corner.

F.6.2 UR edge

Facelet	Color
U	green
R	red

This is the FR edge.

The active pair is therefore recognizable without reconstructing the complete cube.

F.7 The first adjoint query: determining EO for ZBLS

The remaining pair alone does not determine the ZBLS case. We also need the orientation pattern of the four last-layer edges.

The AZBC approach does **not** transport those four edges forward through

$$C, A_1, A_2, A_3.$$

Instead, it forms a small orientation query at x_3 and pulls that query backward to the untouched scramble.

F.7.1 Edge-orientation convention

For each edge position, define a primary facelet using the standard ordered-edge convention:

$$\begin{aligned} UR &= (U, R), & UF &= (U, F), \\ UL &= (U, L), & UB &= (U, B), \\ FR &= (F, R), & LF &= (L, F), \end{aligned}$$

and analogously for the remaining positions.

For an edge cubie, define its distinguished first sticker as:

- Its U or D sticker, when it has one.
- Otherwise its F or B sticker.

The orientation bit is zero exactly when this distinguished sticker occupies the primary facelet of its current position.

For the active pair, $f_\star$ is the ordered sticker-coordinate bit already encoded by $q_\star$, rather than a new color-class inference. Here the remaining FR pair edge at UR has

$$f_\star = 0$$

because its distinguished green sticker lies on the primary U facelet of UR . The white-versus-side-color shortcut below applies only to the four U/D last-layer edge identities.

F.7.2 Choosing the future EO triad

At x_3 , the last-pair edge occupies UR . The four LL edges therefore occupy

$$UL, UB, UF, FR.$$

Choose three of these positions:

$$UL, UB, UF.$$

Their primary facelets have edge Speffz letters

$$\begin{aligned} UL - U &\leftrightarrow D, \\ UB - U &\leftrightarrow A, \\ UF - U &\leftrightarrow C. \end{aligned}$$

Thus the future EO query is the three-letter glyph

$$\boxed{\mathcal{E}_3 = DAC.}$$

The unqueried edge at FR will be recovered from edge-orientation parity.

F.7.3 Pulling the EO triad backward

Let

$$G_3 = A_3 \circ A_2 \circ A_1 \circ C$$

denote the cross-plus-three-pair prefix under the composition convention used throughout this document. We need the preimage

$$\mathcal{E}_0 = G_3^{-1} \mathcal{E}_3.$$

Rather than composing one enormous map, apply each inverse macro successively:

Boundary reached	EO probe word
At x_3 , after A_3	DAC
Pull through A_3^{-1}	ALC
Pull through A_2^{-1}	ALT
Pull through A_1^{-1}	ALT
Pull through C^{-1}	JQO

Therefore,

$$\boxed{G_3^{-1}(DAC) = JQO.}$$

This is the adjoint operation. We transported the **orientation question** backward instead of transporting all LL edges forward.

F.7.4 Reading the three initial probes

On the original scrambled cube, inspect the edge stickers at

$$J, Q, O.$$

The observed colors are

$$\boxed{(J, Q, O) = (\text{white}, \text{white}, \text{green}).}$$

Because the future target positions D, A, C are all primary U -facelets:

- White means that the future LL edge has orientation 0.
- A side color means that it has orientation 1.

Therefore,

$$(f_{UL}, f_{UB}, f_{UF}) = (0, 0, 1).$$

The fourth LL edge lies at FR . Total edge-orientation parity gives

$$f_{\star} \oplus f_{UL} \oplus f_{UB} \oplus f_{UF} \oplus f_{FR} = 0.$$

Substituting

$$f_{\star} = 0, \quad (f_{UL}, f_{UB}, f_{UF}) = (0, 0, 1),$$

we obtain

$$f_{FR} = 0 \oplus 0 \oplus 0 \oplus 1 = 1.$$

Thus, in the positional order

$$(UL, UB, UF, FR),$$

the complete LL-edge orientation word is

$$\boxed{\mathbf{f}_{LL} = 0011.}$$

For verification, the actual five unsolved edges at x_3 are:

Position	Edge occupying it	Orientation
UL	UL	0
UB	UB	0
UF	UF	1

Position	Edge occupying it	Orientation
FR	UR	1
UR	FR , the last-pair edge	0

The planner did not need the first four edge identities. It needed only the three queried bits and parity.

F.8 Selecting the ZBLS algorithm

The ZBLS case key is now

$$\kappa_Z = (q_\star = JB, f_\star = 0, \mathbf{f}_{LL} = 0011).$$

The exact ZBLS card returns

$$Z = R' F R F' U.$$

This five-move macro simultaneously:

1. Solves the FR pair.
2. Preserves the other three F2L pairs.
3. Preserves the cross.
4. Orients every LL edge.

Its pair-state action is

$$JB \xrightarrow{Z} VJ,$$

where VJ is the solved code for the FR pair.

The other solved pair codes remain

$$RB = WN, \quad BL = XR, \quad LF = UF.$$

Thus,

$$(JB, WN, XR, UF) \xrightarrow{Z} (VJ, WN, XR, UF).$$

The EO word changes from

$$0011$$

to

$$0000.$$

After Z , all F2L slots are solved and all LL edges are oriented.

The solved-slot mask is

$$m_4 = (1, 1, 1, 1).$$

F.9 The second adjoint query: determining the future ZBLL

At this point during inspection, Z has been selected but has not physically been executed.

We need to know what last-layer state will exist **after** Z .

Let

$$G_4 = Z \circ G_3$$

denote the complete cross-through- Z prefix.

The naive method would have transported all four LL corners and four LL edges through the complete prefix. AZBC instead asks five carefully selected future-state questions:

3 corner frames + 2 edge identities.

F.9.1 Canonical post-ZBLS sensor basis

At the post-ZBLS state x_4 , choose the corner anchors

$$C, B, A,$$

which are respectively:

$$\begin{aligned} C &= UFR - U, \\ B &= URB - U, \\ A &= UBL - U. \end{aligned}$$

Choose the edge-side probes

$$I, M,$$

where

$$\begin{aligned} I &= UF - F, \\ M &= UR - R. \end{aligned}$$

The canonical future sensor is therefore

$$\boxed{Q_{\text{final}} = CBA \mid IM.}$$

Letters before the bar are corner Speffz positions. Letters after the bar are edge Speffz positions.

Three oriented corners determine the fourth through remaining identity and corner-twist conservation. Two oriented LL edges determine the other two through remaining identity and permutation parity.

F.9.2 The predictor attached to the exact ZBLS algorithm

Every exact ZBLS algorithm has its own predictor:

$$Q_Z = Z^{-1} Q_{\text{final}}.$$

For

$$Z = R'FRF'U,$$

the map is

$$\boxed{CBA \mid IM \xrightarrow{Z^{-1}} IAQ \mid CQ.}$$

Thus the ZBLS algorithm card contains both:

$$\text{motor sequence } R'FRF'U$$

and

$$\text{predictor quintet } IAQ \mid CQ.$$

Changing the ZBLS algorithm would generally change this predictor.

F.9.3 Pulling the predictor through the F2L prefix

Now pull

$$IAQ \mid CQ$$

backward through A_3, A_2, A_1, C :

Boundary reached	Corner probes	Edge probes
Post- Z : Q_{final}	CBA	IM
Through Z^{-1}	IAQ	CQ
Through A_3^{-1}	LBR	CF
Through A_2^{-1}	LOR	TF
Through A_1^{-1}	LOS	TF
Through C^{-1}	DLC	OA

Therefore the complete initial-state predictor is

$$Q_0 = DLC \mid OA.$$

Equivalently,

$$G_3^{-1} \circ Z^{-1} Q_{\text{final}} = DLC \mid OA.$$

Only these five initial cubie frames/stickers must be inspected to identify the exact future ZBLL.

F.10 Reading the five ZBLL probes

F.10.1 Corner probe D

Corner letter D is the U -face sticker of the current ULF position.

The corner at that position has colors

Face	Color
U	orange
L	white
F	blue

so it is the UBL corner.

The marked sticker at D is orange.

Because

$$D \xrightarrow{G_4} C,$$

this tells us:

After ZBLS, the UBL corner will occupy UFR , and its orange sticker will be on the U face.

Under the chosen corner-orientation convention, this is twist 2.

F.10.2 Corner probe L

Corner letter L is the F -face sticker of the current DLF position.

The corner at that position has colors

Face	Color
D	orange
L	white
F	green

so it is the ULF corner.
 The marked L sticker is green.
 Because

$$L \xrightarrow{G_4} B,$$

this tells us:

After ZBLS, the ULF corner will occupy URB , with green on the U face.

This is also twist 2.

F.10.3 Corner probe C

Corner letter C is the U -face sticker of the current UFR position.
 The corner at that position has colors

Face	Color
U	red
R	green
F	white

so it is the UFR corner.
 The marked C sticker is red.
 Because

$$C \xrightarrow{G_4} A,$$

this tells us:

After ZBLS, the UFR corner will occupy UBL , with red on the U face.

This is twist 2.

F.10.4 Edge probe O

Edge letter O is the R -face sticker of the current DR position.
 The edge at that position is the UF edge, and the marked sticker is green.
 Because

$$O \xrightarrow{G_4} I,$$

and I is the F -side sticker at UF , we conclude:

After ZBLS, UF contains the UF edge.

Thus

$$E(0) = 0.$$

F.10.5 Edge probe A

Edge letter A is the U -face sticker of the current UB position.

The edge there is the UB edge, and the marked sticker is blue.

Because

$$A \xrightarrow{G_4} M,$$

and M is the R -side sticker at UR , we conclude:

After ZBLS, UR contains the UB edge.

Thus

$$E(1) = 2.$$

F.11 Reconstructing the complete corner state

Index the post-ZBLS sectors as

$$\begin{aligned} 0 &: UFR \leftrightarrow UF, \\ 1 &: URB \leftrightarrow UR, \\ 2 &: UBL \leftrightarrow UB, \\ 3 &: ULF \leftrightarrow UL. \end{aligned}$$

Use the same indices for corner identities:

$$0 = UFR, \quad 1 = URB, \quad 2 = UBL, \quad 3 = ULF.$$

From the three corner probes:

$$\begin{aligned} UFR &\leftarrow UBL, \\ URB &\leftarrow ULF, \\ UBL &\leftarrow UFR. \end{aligned}$$

The remaining identity is URB , so

$$ULF \leftarrow URB.$$

Therefore the corner position-to-identity permutation is

$$C = [2, 3, 0, 1].$$

In cycle notation,

$$C = (0\ 2)(1\ 3).$$

It is an even permutation.

The observed corner twists are

$$2, 2, 2.$$

Corner-twist conservation requires

$$\tau_0 + \tau_1 + \tau_2 + \tau_3 \equiv 0 \pmod{3}.$$

Therefore,

$$2 + 2 + 2 + \tau_3 \equiv 0 \pmod{3}.$$

Since

$$2 + 2 + 2 = 6 \equiv 0 \pmod{3},$$

we obtain

$$\tau_3 = 0.$$

The complete corner-orientation vector is

$$\tau = [2, 2, 2, 0].$$

Thus the post-ZBLS corner chunk is

$$K_C = ([2, 3, 0, 1], [2, 2, 2, 0]).$$

F.12 Reconstructing the complete edge permutation

Use the paired edge positions

$$(UF, UR, UB, UL)$$

with identities

$$0 = UF, \quad 1 = UR, \quad 2 = UB, \quad 3 = UL.$$

The two edge probes give

$$E(0) = 0, \quad E(1) = 2.$$

Therefore the first two entries are

$$E = [0, 2, _, _].$$

The remaining identities are

$$\{1, 3\}.$$

There are only two possible completions:

$$E_a = [0, 2, 1, 3]$$

or

$$E_b = [0, 2, 3, 1].$$

The first completion is the transposition

$$(1\ 2),$$

which is odd.

The second completion is the three-cycle

$$(1\ 2\ 3),$$

which is even.

The corner permutation C is even, and legal cube states require corner and edge permutation parity to agree. Therefore,

$$E = [0, 2, 3, 1].$$

For verification, the post-ZBLS LL edges are:

Position	Edge identity
UF	UF
UR	UB
UB	UL
UL	UR

All are oriented.

F.13 Computing the A_4 edge token

The LL edge state is represented relative to the corner frame by

$$\rho = C^{-1} \circ E.$$

Here,

$$C = [2, 3, 0, 1].$$

This permutation is its own inverse:

$$C^{-1} = [2, 3, 0, 1].$$

Apply C^{-1} to each entry of E :

$$\begin{aligned}\rho(0) &= C^{-1}(E(0)) = C^{-1}(0) = 2, \\ \rho(1) &= C^{-1}(E(1)) = C^{-1}(2) = 0, \\ \rho(2) &= C^{-1}(E(2)) = C^{-1}(3) = 1, \\ \rho(3) &= C^{-1}(E(3)) = C^{-1}(1) = 3.\end{aligned}$$

Therefore,

$$\boxed{\rho = [2, 0, 1, 3].}$$

In cycle notation,

$$\rho = (0\ 2\ 1).$$

Because an element of A_4 is uniquely determined by the images of its first two objects, the two-symbol code is

$$\boxed{R_2 = \rho(0)\rho(1) = 20.}$$

The exact ZBLL state is therefore indexed by

$$\boxed{(K_C, R_2) = ([2, 3, 0, 1], [2, 2, 2, 0]),\ 20) .}$$

This is the holistic last-layer recognition result:

- One corner chunk.
- One tetrahedral A_4 token.

No four-edge permutation memo is needed.

F.14 Selecting ZBLL

The lookup

$$([2, 3, 0, 1], [2, 2, 2, 0], 20) \mapsto L$$

returns

$$L = RU R' U R U^2 R'.$$

This is a Sune-family seven-move ZBLL for this exact permutation and orientation state.

In this constructed case, applying the sequence does not merely orient the corners—it solves the entire cube:

$$L(x_4) = e.$$

No pre-AUF or post-AUF is required in the chosen frame.

F.15 The complete compiled solve

The full motor program is

$$\mathcal{P} = [F D R' B L^2 D'] \\ [U L U' L'] \\ [U' R' U R] \\ [U' L' U L] \\ [R' F R F' U] \\ [R U R' U R U^2 R'].$$

As one sequence:

F D R' B L2 D'
U L U' L'
U' R' U R
U' L' U L
R' F R F' U
R U R' U R U2 R'

Or one line:

F D R' B L2 D' U L U' L' U' R' U R U' L' U L R' F R F' U R U R' U R U2 R'

F.16 Execution checkpoints

The timed solve is not intended to include any new search or case selection. Vision is used only to confirm predicted checkpoints.

Chunk	Predicted checkpoint
C	Yellow cross solved; pair ring begins JR, QM, RD, II
A_1	BL solved; $RB = QM, FR = FC, LF = JE$
A_2	RB, BL solved; $FR = JB, LF = IE$
A_3	Three pairs solved; final pair $FR = JB$
Z	All four F2L pairs solved; EO solved
L	Cube solved

The actual solve is therefore a self-indexing chain:

$$C \xrightarrow{x_C} A_1 \xrightarrow{x_1} A_2 \xrightarrow{x_2} A_3 \xrightarrow{x_3} Z \xrightarrow{x_4} L.$$

Each physical state supplies a confirmation cue for the next compiled chunk.

F.17 What was held in working memory

The exact state-register schedule is:

F.17.1 After the cross is selected

Four pair codes:

$$(JR, QM, RD, II),$$

or eight letters.

F.17.2 After A_1 is compiled

Discard the solved BL pair:

$$(FC, QM, JE),$$

or six letters.

F.17.3 After A_2 is compiled

Discard the solved RB pair:

$$(JB, IE),$$

or four letters.

F.17.4 After A_3 is compiled

Only the ZBLS pair remains:

$$JB,$$

or two letters.

F.17.5 EO query

Temporarily carry the triad

$$DAC$$

backward to

$$JQO.$$

Its observed colors generate the EO token

$$0011.$$

The letters can then be discarded.

F.17.6 ZBLL query

Temporarily carry the predictor

$$CBA \mid IM$$

backward to

$$DLC \mid OA.$$

Its observations generate the corner chunk and A_4 token

$$([2, 3, 0, 1], [2, 2, 2, 0], 20).$$

The probe letters can then be discarded.

The symbolic liveness profile is therefore

$$\boxed{8 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 5.}$$

The final retained object is not a cube-state memo. It is the six-chunk motor program

$$[C][A_1][A_2][A_3][Z][L].$$

F.18 Probe overlap in this solve

The initial EO probes are

$$\{J, Q, O\}.$$

The initial ZBLL probes are

$$\{D, L, C, O, A\}.$$

They share probe O . Therefore the number of unique late-stage sticker positions is

$$|\{J, Q, O\} \cup \{D, L, C, O, A\}| = 7,$$

not eight.

The green value observed at O during EO acquisition can be reused during ZBLL acquisition.

This is a concrete example of **inspection-aware algorithm selection**. A mature AZBC algset would favor ZBLS algorithms whose EO and ZBLL predictor probes overlap or remain spatially clustered.

F.19 The complete inspection computation in compact form

The entire worked example can be condensed to the following compiler trace.

F.19.1 Forward F2L pass

$$\begin{aligned} Q_{\text{initial}} &= (BK, PN, HW, RL), \\ Q_0 &= (JR, QM, RD, II), \\ A_1 : \quad Q_1 &= (FC, QM, XR, JE), \\ A_2 : \quad Q_2 &= (JB, WN, XR, IE), \\ A_3 : \quad Q_3 &= (JB, WN, XR, UF). \end{aligned}$$

The remaining pair is

$$q_\star = JB.$$

F.19.2 Adjoint EO pass

$$DAC \xrightarrow{A_3^{-1}} ALC \xrightarrow{A_2^{-1}} ALT \xrightarrow{A_1^{-1}} ALT \xrightarrow{C^{-1}} JQO.$$

Read

$$JQO = (W, W, G),$$

giving

$$\mathbf{f}_{LL} = 0011.$$

Select

$$Z = R'FRF'U.$$

F.19.3 Adjoint ZBL pass

$$CBA \mid IM \xrightarrow{Z^{-1}} IAQ \mid CQ \xrightarrow{A_3^{-1}} LBR \mid CF$$

$$\xrightarrow{A_2^{-1}} LOR \mid TF \xrightarrow{A_1^{-1}} LOS \mid TF \xrightarrow{C^{-1}} DLC \mid OA.$$

Read the five initial probes and decode

$$C = [2, 3, 0, 1], \quad \tau = [2, 2, 2, 0], \quad R_2 = 20.$$

Select

$$L = RUR'URU^2R'.$$

F.19.4 Output program

$$\boxed{[C][A_1][A_2][A_3][Z][L].}$$

F.20 What full mastery would change

The verbose derivation above is an audit trail. A mastered solver would not consciously narrate:

“Corner R maps to X , edge D maps to R , therefore RD becomes XR .”

Instead,

$$(JR, QM, RD, II) \xrightarrow{A_1} (FC, QM, XR, JE)$$

would be perceived as one holistic constellation transformation.

Likewise, the solver would not independently transform five predictor letters. The glyph

$$CBA \mid IM$$

would be seen flowing backward through the exact ZBLS and F2L macros until it lands on

$$DLC \mid OA.$$

The mature mental objects would be:

four-pair constellation,
three-point EO glyph,
five-point predictor glyph,
corner-state chunk,
tetrahedral edge token 20,
six linked motor chunks.

This solve demonstrates the defining idea of AZBC:

Propagate only F2L control state forward, pull future LL questions backward.

The method never requires the solver to carry the complete last-layer state through all four F2L stages. It uses the original scrambled cube as external read-only memory, queries exactly the information that becomes live, and compiles the result into an uninterrupted executable solution.